ITLKPFU
Казанский федеральный университет
СУНЦ КФУ IT-лицей
НАУКА И УНИВЕРСИТЕТЫ
НАЦИОНАЛЬНЫЕ ПРОЕКТЫ РОССИИ

Д.Н. ТУМАКОВ, Д.Х. ГИНИЯТОВА, В.С. ГУСЬКОВ

ПРОГРАММИРОВАНИЕ ГРАФИЧЕСКИХ УСТРОЙСТВ

ТЕХНОЛОГИЯ CUDA

МЕТОДИЧЕСКАЯ БИБЛИОТЕКА

КАЗАНСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ

**Специализированный учебный научный центр –
общеобразовательная школа-интернат «IT-лицей»**

Д.Н. ТУМАКОВ, Д.Х. ГИНИЯТОВА, В.С. ГУСЬКОВ

ПРОГРАММИРОВАНИЕ ГРАФИЧЕСКИХ УСТРОЙСТВ. ТЕХНОЛОГИЯ CUDA

Учебное пособие



**КАЗАНЬ
2022**

УДК 004.4:681.3
ББК 32.81
Т83

Печатается по рекомендации ученого совета
СУНЦ IT-лицей КФУ
(протокол № 2 от 25.09.2022 г.)

Авторы:
заместитель директора по научной деятельности Института вычислительной математики и информационных технологий КФУ, кандидат физико-математических наук, доцент
Тумаков Д.Н.
старший преподаватель кафедры прикладной математики и искуственного интеллекта Института вычислительной математики и информационных технологий КФУ
Гиниятова Д.Х.
заместитель директора СУНЦ IT-лицей КФУ
Гуськов В.С.

Научный редактор
доктор физ.-мат. наук, проф. **Н.Б. Плещинский**

Рецензенты:
кандидат физ.-мат.наук, доцент **И.Е. Плещинская**
кандидат физ.-мат.наук, доцент **Е.В. Рунг**

Тумаков Д.Н.

Т83 Программирование графических устройств. Технология CUDA: [Электронный ресурс]: учебное пособие / Д.Н. Тумаков, Д.Х. Гиниятова, В.С. Гуськов. – Электрон. текстовые дан. (1 файл: 1,70 Мб). – Казань: Издательство Казанского университета, 2022. – 86 с. – Систем. требования: Adobe Acrobat Reader. – URL: https://kpfu.ru/portal/docs/F1508134442/Tumakov_Programmirovanie_graficheskikh_ustrojstv._1_.2.pdf. – Загл. с титул. экрана.

ISBN 978-5-00130- 665-8

В пособии описаны технология CUDA и основные принципы работы с ней. Рассмотрены вопросы использования разделяемой, константной и текстурной памяти. Приведены примеры реализации различных алгоритмов. В конце глав содержатся упражнения для лабораторных и домашних занятий.

Пособие предназначено для обучающихся старших классов, изучающих информатику и математику на профильном уровне.

УДК 004.4:681.3
ББК 32.81

ISBN 978-5-00130-665-8

© Издательство Казанского университета, 2022
© Тумаков Д.Н., Д.Х. Гиниятова, В.С. Гуськов, 2022

Оглавление

Оглавление 3
Введение 5
Глава 1. Модель программирования CUDA 7
Основные понятия 7
Сетки, блоки, нити 7
Варпы 9
Архитектура GPU 10
Архитектура видеокарт NVidia 11
Типы видеокарт, поддерживающих CUDA 15
Получение информации об устройстве 16
Глава 2. Расширение языка CUDA C 21
Первая программа 21
Спецификаторы функций и переменных 22
Встроенные типы данных и встроенные переменные 24
Директивы вызова ядра 26
Глава 3. Особенности написания программ на CUDA 29
Работа с памятью 29
Сложение двух векторов 31
Типичная структура CUDA программы 32
Обработка ошибок 32
Замер времени работы части кода программы 33
Глава 4. Атомарные операции в CUDA 37
Атомарные арифметические операции 37
Атомарные побитовые операции 39
Дополнительные возможности Compute Capability 6.x 40
Пример использования атомарных операций 41
Глава 5. Работа с векторами. Математические функции 43
Сложение векторов 43
Вычисление математических функций 45
Глава 6. Работа с матрицами 49
Создание матриц 50
Транспонирование матрицы 53
Сложение матриц 58
Умножение матриц 59
Глава 7. Типы памяти. Разделяемая память 66
Банки данных 69

Пример. Оптимизация программы по перемножению двух матриц 71
Глава 8. Константая и текстурная память 77
Константная память 77
Текстурная память 78
Примеры использования одномерной текстуры 79
Литература 85

ВВЕДЕНИЕ

Параллельные вычисления давно уже перестали быть специальными технологиями, которыми пользовались лишь в исследовательских центрах. Сегодня методы параллельных вычислений в той или иной степени используются при решении практически всех сложных задач. Совершенствование технической базы компьютеров в сочетании с параллельной обработкой данных позволило существенно повысить производительность вычислений.

В данном пособии рассмотрим одну из технологий высокопроизводительных вычислений, использующую графический процессор (GPU). Изначально этот класс устройств разрабатывался для обработки графики. Техника совершенствовалась, GPU наращивали производительность, и в какой-то момент оказалось, что GPU можно успешно использовать не только для задач компьютерной графики, но и как математический сопроцессор для CPU, получая при этом существенный прирост производительности.

Этот класс технологий получил название GPGPU (General-purpose computing for graphics processing units) – использование графического процессора для различных вычислительных задач. Популярный представитель этого класса – CUDA (Compute Unified Device Architecture), который был впервые представлен компанией NVIDIA в 2007 году. CUDA может использоваться на GPU производства NVIDIA, таких как GeForce, Quadro, Tesla, Fermi, Kepler, Maxwell, Pascal и Volta. Некоторые из этих устройств не имеют видеовыхода, и эти видеокарты предназначены исключительно для высокопроизводительных вычислений.

CUDA работает только с устройствами производства NVIDIA, но отметим, что помимо CUDA существуют и другие аналогичные технологии, например, OpenCL, AMD FireStream, CUDAfy и PyCUDA.

В качестве дополнительной литературы можно порекомендовать [1–3]. В [1] разобрана как сама технология CUDA, так и вопросы оптимизации кода с использованием ptx-ассемблера. В книге также рассмотрены задачи по

моделированию ряда физических процессов. Задачи моделирования и использование стандартных библиотек рассмотрены и в [2]. В [3] подробно описана работа с несколькими графическими устройствами.

Глава 1. Модель программирования CUDA

В первой главе рассмотрим базовые понятия технологии CUDA – хост, устройство, ядро, сетка, блок и нить, познакомимся с архитектурой графического процессора и узнаем, как определить основные характеристики GPU, установленные на вашем компьютере.

Основные понятия

Программный код в CUDA состоит как из последовательных, так и из параллельных частей. Последовательные части кода выполняются на ***хосте (host)*** – центральном процессоре (CPU), а массивно-параллельные части кода выполняются на ***устройстве (device)*** – графическом процессоре (GPU), расположенном на видеокарте. Каждый процессор имеет свою отдельную память, память хоста и память устройства физически разделены. Хост управляет вычислительным процессом – с хоста вызываются функции, организующие обмен данными между различными видами памяти, и запускаются задачи на устройстве. Каждой такой задаче соответствует **ядро *(kernel)*** – функция, оформленная по особым правилам.

В настоящее время технологию CUDA поддерживают такие языки программирования, как C/C++, Fortran и Phyton. В программу, написанную для CPU, можно добавить дополнительные элементы для работы с GPU. Компилятор CUDA работает как препроцессор. После предварительной обработки исходного кода вызывается стандартный компилятор и генерируется исполняемый код отдельно для хоста и отдельно для устройства.

Сетки, блоки, нити

При вычислениях на GPU одновременно запускается большое число параллельных процессов, их принято называть ***нитями (thread)***. Все запущенные на выполнение нити объединены в сложную структуру – ***сетку (grid).*** Сетка представляет собой одномерный, двухмерный или трехмерный массив ***блоков (block)***. Каждый блок – это одномерный, двухмерный или трехмерный массив

нитей (thread). При этом все блоки, образующие сетку, имеют одинаковую размерность и размер (рис. 1).

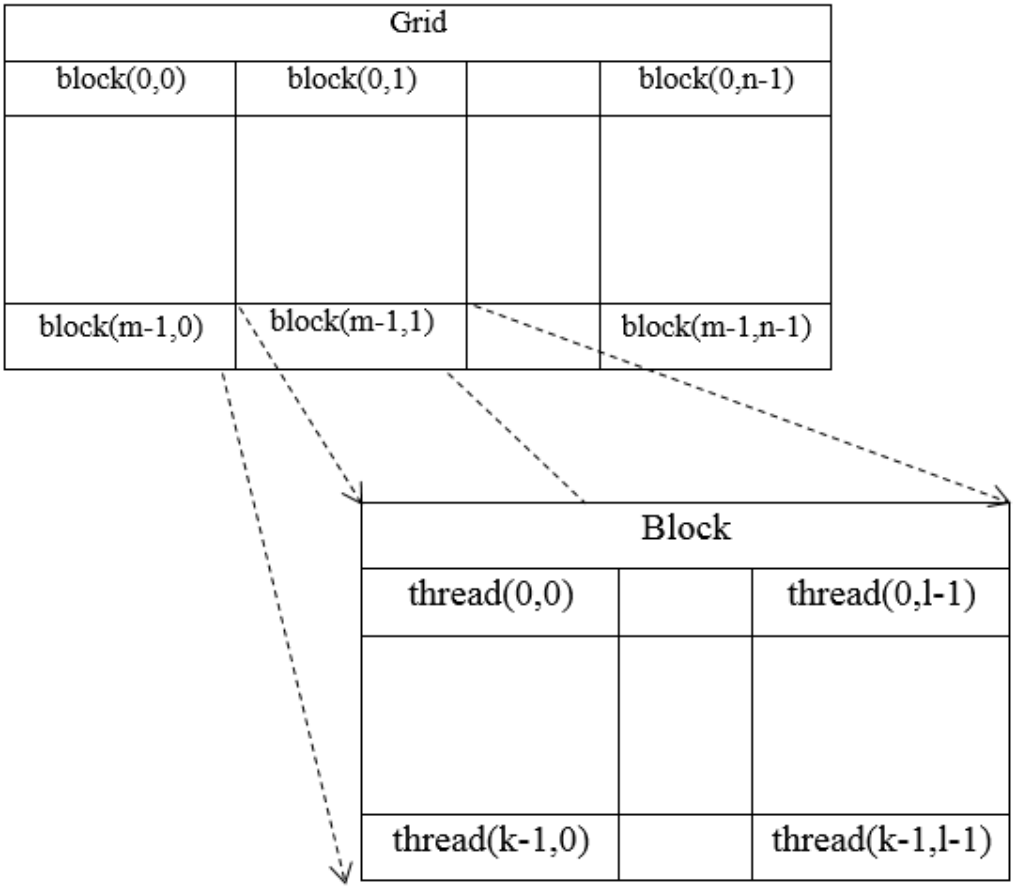


Рис. 1. Иерархия нитей в CUDA

Подобное разделение всех нитей является еще одним общим приемом использования CUDA: исходная задача разбивается на набор отдельных подзадач, решаемых независимо друг от друга (рис. 2). Каждой такой подзадаче соответствует блок нитей. При этом каждая подзадача совместно решается всеми нитями своего блока. Объединение нитей в варпы происходит отдельно для каждого блока; таким образом, все нити одного варпа всегда принадлежат одному блоку. При этом нити могут взаимодействовать между собой только в пределах блока. Нити разных блоков взаимодействовать между собой не могут.

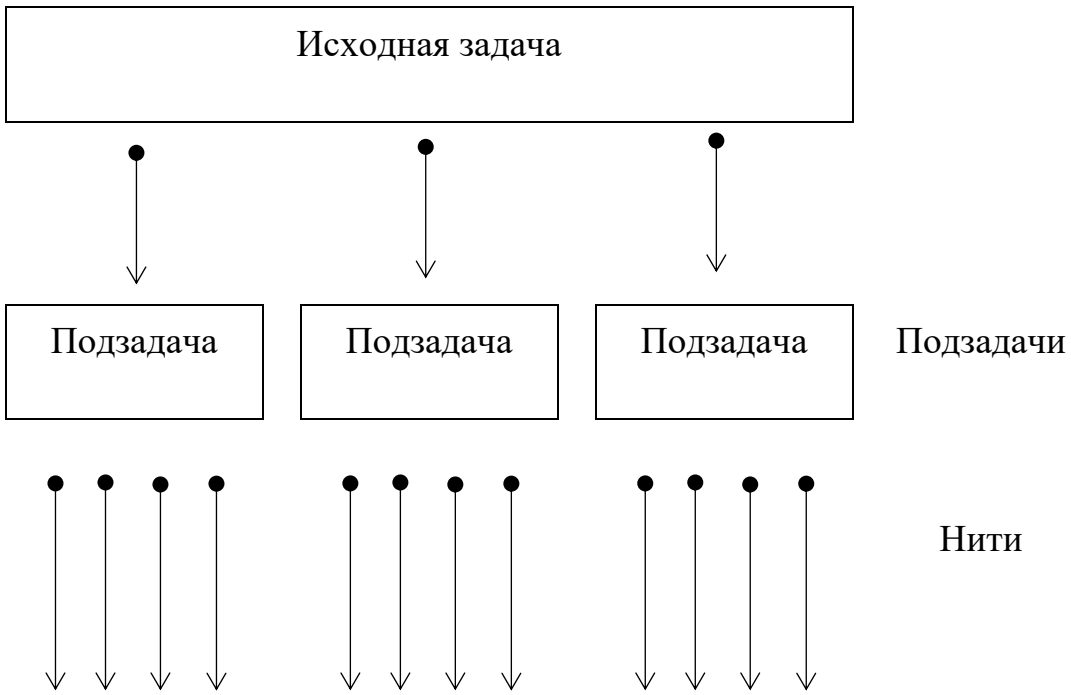


Рис. 2. Разбиение исходной задачи на набор независимо решаемых подзадач

Подобный подход является удачным компромиссом между необходимостью обеспечить взаимодействие нитей между собой и стоимостью подобного взаимодействия – обеспечить возможность взаимодействия каждой нити с каждой было бы слишком сложно и дорого.

Однако есть способы косвенного взаимодействия нитей в блоке. Они могут взаимодействовать друг с другом через разделяемую (shared) память (об использовании разделяемой памяти речь пойдет в главе 7).

Варпы

Варп (Warp) – группировка потоков по 32 штуки, которые оказываются частями более крупных образований – блоков (blocks). Все потоки каждого блока запускаются строго на одном потоковом мультипроцессоре (SM, Streaming Multiprocessors), поэтому имеют доступ только к его ресурсам. Однако на одном SM может запускаться более одного блока, и тогда ресурсы будут разделяться между ними поровну.

В каждом SM существует блок управления, который занимается распределением ресурса процессорного времени. Делается это так, что в каждый момент времени все ядра одного SM исполняют строго один варп. По его завершении работы варпа оптимальным способом выбирается следующий варп, приписанный к данному SM. Таким образом, оказывается, что потоки одного варпа синхронизируются за счет аппаратной особенности CUDA и исполняются по еще более близкому к SIMD (Single Instruction, Multiple Data) методу. Но потоки даже одного блока из разных варпов могут оказаться заметно рассинхронизированными.

Можно сделать вывод, что взаимодействие между потоками одного блока следует стараться осуществлять через их общую быструю разделяемую память, а между потоками двух различных блоков – только с использованием глобальной памяти. Тут возникает проблема: слежение за актуальностью данных в общедоступной разным потокам для чтения и записи области памяти. Иначе говоря – проблема синхронизации потоков. Как уже отмечалось, в рамках одного блока потоки каждого варпа синхронизированы между собой. Для синхронизации потоков блока вне зависимости от принадлежности к варпам существует несколько команд барьерного типа (о них подробнее расскажем в заключительном разделе данной главы «Синхронизация»).

Архитектура GPU

GPU построен как масштабируемый массив потоковых мультипроцессоров (SM, Streaming Multiprocessor). Когда на CUDA запускается ядро на выполнение, то блоки сетки выполняются на имеющихся мультипроцессорах. При этом каждый блок целиком выполняется на одном из мультипроцессоров, который, в свою очередь, способен одновременно выполнять до восьми блоков. По мере того как отдельные блоки завершают свое выполнение, на их место становятся новые блоки. Поэтому даже на довольно небольшом числе потоковых мультипроцессоров можно запустить на выполнение сетку с большим числом блоков. Многопоточная программа распределяется по блокам нитей, которые выполня-

ются независимо друг от друга, поэтому GPU с большим числом ядер выполнит программу за меньшее время, чем GPU с меньшим числом ядер (рис. 3). Для обозначения возможностей GPU CUDA использует понятие Compute Capability (cc), выражаемое парой чисел – major.minor. Первое число обозначает глобальную архитектурную версию, второе – небольшие изменения в архитектуре.

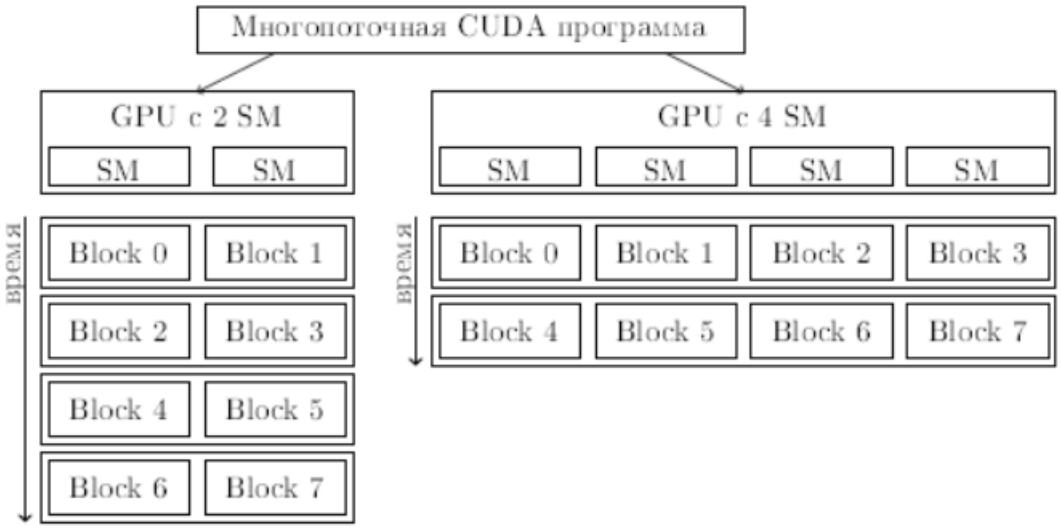


Рис. 3. Масштабируемая программная модель

Для написания оптимальных программ на графических процессорах важно знать особенности архитектуры устройства, под которое будет написан код.

Архитектура видеокарт NVidia

Рассмотрим подробнее физическое устройство видеокарт. На рис. 4 приведена схема архитектуры видеокарты GeForce 8800 GT. Центральный процессор обменивается данными с GPU через мост (Bridge). Вычислительная часть видеокарты состоит из семи текстурных процессорных кластеров (Texture Processor Cluster), каждый из которых содержит два мультипроцессора (SM), блок текстур (Texture Unit) и совмещенный текстурный кэш и кэш первого уровня (Tex/L1).

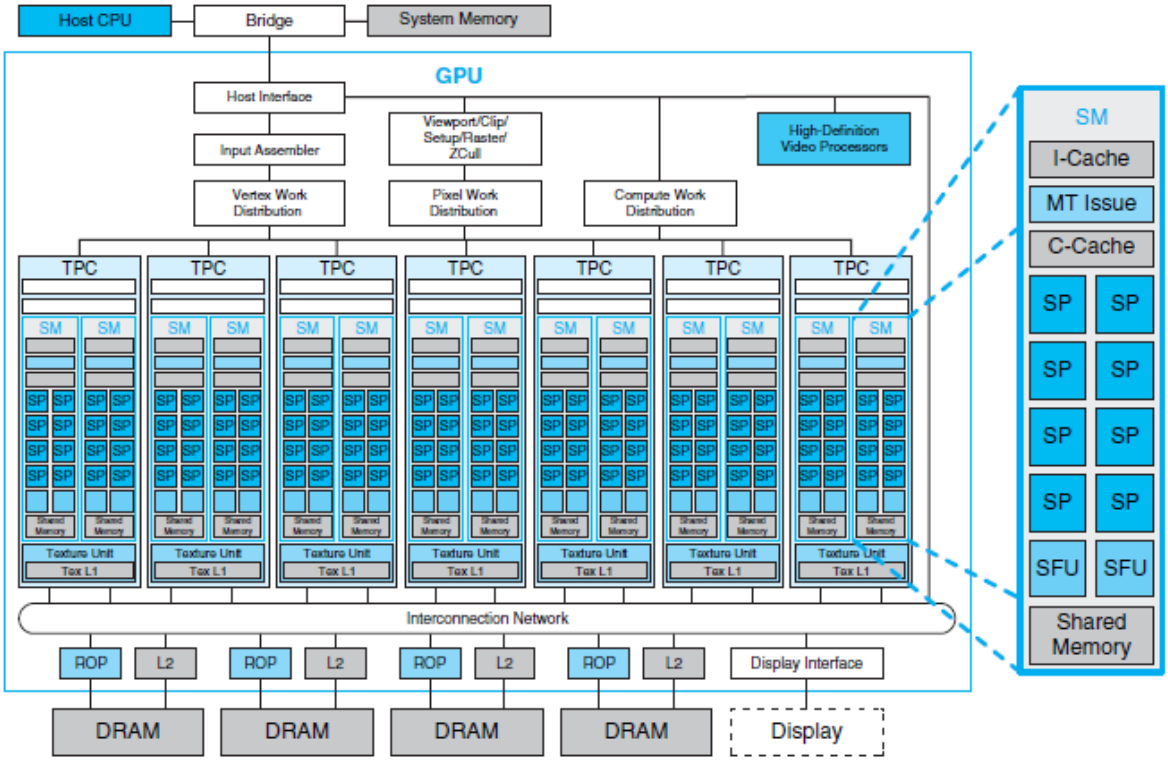


Рис. 4. Архитектура видеокарты GeForce 8800 GT

Блоки текстур осуществляют выборку и фильтрацию текстурных данных, необходимых для построения сцены. Число текстурных блоков в видеочипе определяет текстурную производительность и скорость выборки из текстур. Текстурная производительность является одним из важнейших параметров видеочипов. Особое влияние этот параметр оказывает на скорость при использовании трилинейной и анизотропной фильтраций, требующих дополнительных текстурных выборок.

Мультипроцессор, в свою очередь, содержит 8 потоковых процессоров (Streaming Processors) или ядер CUDA (CUDA cores). Таким образом, общее число SP = 14 * 8 = 112. Также каждый SM содержит по два блока для вычисления специальных (трансцендентных) функций (SFU), таких как синус, косинус и т. д. В состав SM входит разделяемая память (Shared Memory), блок инструкций нитей (Multithreaded Instruction Unit), константный кэш (C-Cache) и кэш инструкций (L-Cache).

Сравнение базовых характеристик видеокарт серии GF 8800 приведено в табл. 1.

Таблица 1

Сравнительные характеристики видеокарт GF 8800

	GeForce 8800 Ultra	**GeForce 8800 GTX**	**GeForce 8800 GTS**	**GeForce 8800 GT**	**GeForce 8800 GS**
Stream процессоры	128	128	96	112	96
Частота ядра (МГц)	612	575	500	600	550
Частота шейдерного блока (МГц)	1500	1350	1200	1500	1375
Частота памяти (МГц)	1080	900	800	900	800
Объем памяти	768MB	768MB	640MB и 320MB	512MB	384MB
Интерфейс памяти	384-bit	384-bit	320-bit	256-bit	192-bit
Полоса пропускания памяти (ГБ/с)	103.7	86.4	64	57.6	38.4

Вычислительная часть устройства соединяется с видеопамятью (DRAM) посредством кэша второго уровня (L2) и блоков операций растеризации (ROP). Блоки операций растеризации осуществляют операции записи рассчитанных видеокартой пикселей в буферы и операции их смешивания (блендинга). Производительность блоков ROP влияет на филлрейт (скорость заполнения пикселами), а это является одной из основных характеристик видеокарт.

Надо иметь в виду, что видеокарты одной линейки имеют одинаковую производительность, и, следовательно, одинаковое число SP в MP. Однако остальные характеристики могут отличаться. В таблице 2 приведены характеристики для GeForce 8800.

Рассмотрим также новый, недавно анонсированный, мультипроцессор семейства Volta (рис. 5). Каждый SM содержит четыре блока, в которых есть помимо ядер, выполняющих целочисленные вычисления, ядра для работы с вещественными 32 и 64-битными числами, а также новые, появившиеся

в данной линейке процессоров, тензорные ядра (Tensor Cores), разработанные специально для обучения нейронных сетей в задачах глубокого обучения.

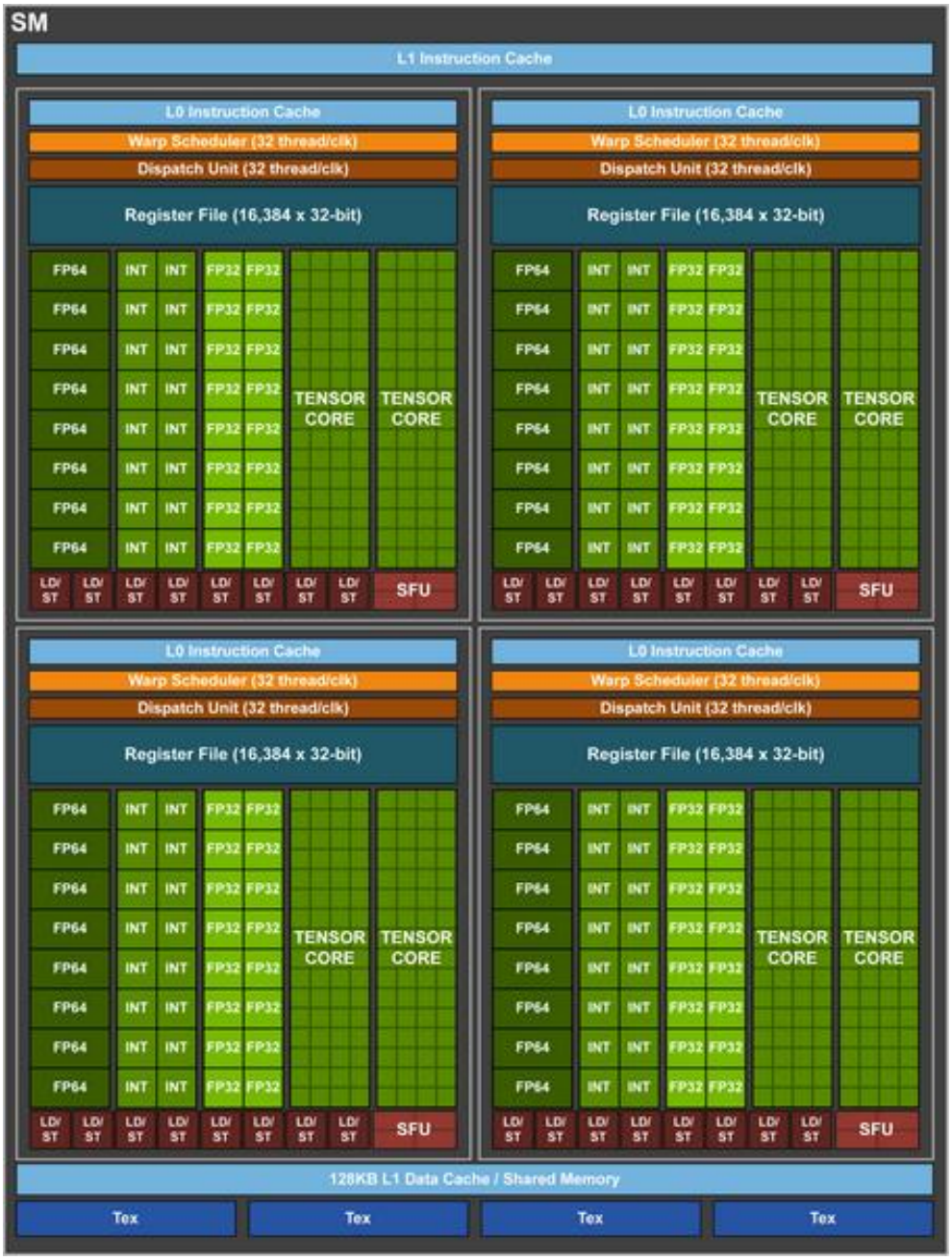


Рис. 5. Архитектура мультипроцессора семейства Volta GV100

На рисунке 5 таже представлены планировщик варпов (Warp Scheduler), блок выборки (Dispatch Unit), регистры мультипроцессора (Register File) и блоки загрузки и сохранения (LD/ST – Load/Store Units).

Типы видеокарт, поддерживающих CUDA

Приведем в табл. 2 характеристики некоторых видеокарт, поддерживающих технологию CUDA.

Первая колонка таблицы – название видеокарты: указывается модель и объем памяти, в названии GT обозначает обычные видеокарты низкого уровня производительности, GTX обозначает видеоадаптеры среднего и высокого уровня, Ti или Titan обозначает десктопные карты.

Вторая колонка обозначает маркировку кристаллов: GT – Tesla, GF – Fermi, GK – Kepler, GM – Maxwell, GP – Pascal. Третья и четверая колонки фактически показывают производительность видеокарты. Цифры в последней колонке, разрядности шины памяти, указывают на пропускную способность памяти.

Отметим, что частота ядра, как правило, немного ниже частот шейдерных блоков. Частота же памяти в современных видеокартах существенно выше частоты ядра.

Таблица 2

Характеристики некоторых видеокарт

GeForce	GPU / кристалл	Кол-во процессоров-ядер / текстурных блоков / ROP – блоков растеризации	Частота ядра GPU / шейдерного блока / памяти в MHz.	Разрядность шины пямяти
Titan X 12gb	GP102	3584/224/96	1417/1530/10000	384 bit
GTX 1080 8gb	GP104	2560/160/64	1607/1733/10000	256 bit
GTX 1060 6gb	GP106	1280/80/48	1506/1709/8000	192 bit
GTX 980 Ti 4gb	GM200	2816/176/96	1000/1075/7000	384 bit
GTX 980 4 gb	GM204	2048/104/64	1126/1216/7000	256 bit
GTX 960 2 gb	GM206	1024/64/32	1126/1178/7000	128 bit
Titan Z 12gb	2xGK110	5760/448/96	705/876/7000	2×384
GTX 780 Ti 3gb	GK110	2880/240/48	875/928/7000	384 bit
GTX 760 2 gb	GK104	1152/96/32	980/1033/6008	256 bit
GTX 750 1 gb	GM107	512/32/16	1020/1085/5000	128 bit
GTX 680 2 gb	GK104	1538/128/32	1006/1058/6008	256 bit
GTX 660 2 gb	GK106	960/80/24	980/1033/6008	192 bit
GT 640 2 gb	GK107	384/32/16	900/900/1784	128 bit
GT 630 1 gb	GF108	96/16/4	810/1620/3200	128 bit

Продолжение табл. 2

GeForce	GPU / кристалл	Кол-во процессоров-ядер / текстурных блоков / ROP – блоков растеризации	Частота ядра GPU / шейдерного блока / памяти в MHz.	Разрядность шины пямяти
GT 610 1 gb	GF119	48/8/4	810/1620/1800	64 bit
GTX 580 1.5 gb	GF110	512/64/48	772/1544/4008	384 bit
GTX 480 1.5 gb	GF100	480/60/48	701/1402/3700	384 bit
GTX 560 1 gb	GF114	336/56/32	675/1620/4004	256 bit
GTX 460 1 gb	GF104	336/56/32	675/1350/3600	256 bit
GTX 550 Ti 1gb	GF116	192/32/24	900/1800/4100	192 bit
GTS 450 1 gb	GF106	192/32/16	783/1566/3600	128 bit
GT 440 1 gb	GF108	96/16/4	810/1620/3200	128 bit
GT 520 1 gb	GF119	48/8/4	810/1620/1800	64 bit
GTX 280 1 gb	GT200	240/80/32	602/1296/2215	512 bit
GTS 250 1 gb	G92b	128/64/16	738/1836/2200	256 bit
GT 240 1 gb	GT215	96/32/8	550/1340/3400	128 bit
GT 220 1 gb	GT216	48/16/8	625/1360/1580	128 bit
GT 210 1 gb	GT218	16/8/4	589/1402/1000	64 bit
9800 GT 1 gb	G92	112/56/16	600/1500/1800	256 bit
9600 GT 1 gb	G94	64/32/16	650/1625/1800	128 bit
9500 GT 1 gb	G96	32/16/8	550/1400/1600	128 bit
8400 GS 512 mb	G86	16\8\4	450/900/800	64 bit

Получение информации об устройстве

Получим от видеокарты информацию о ее возможностях и разберемся, что они дают. Для определения свойств видеокарты используется функция cudaGetDeviceProperties(), принимающая в качестве параметра номер видеокарты и возвращающая в структуре cudaDeviceProp интересующие нас значения. Выведем на экран полученные переменные.

Приведем основные переменные структуры cudaDeviceProp:

```
struct cudaDeviceProp {
        char name[256];
        size_t totalGlobalMem;
        size_t sharedMemPerBlock;
        int regsPerBlock;
        int warpSize;
        size_t memPitch;
```

```
            int maxThreadsPerBlock;
            int maxThreadsDim[3];
            int maxGridSize[3];
            size_t totalConstMem;
            int clockRate;
            int deviceOverlap;
            int multiProcessorCount;
            int kernelExecTimeoutEnabled;
            int integrated;
            int canMapHostMemory;
            int concurrentKernels;
    …
        }
```

Здесь:

- name[256] представляет собой ASCII строку идентификации устройства;
- totalGlobalMem – общий объем глобальной памяти, доступной на устройстве в байтах;
- sharedMemPerBlock – максимальный объем общей памяти, доступной для нити блока в байтах;
- regsPerBlock – максимальное количество 32-битных регистров, доступных для нити блока;
- warpSize – размер Варпа в потоках;
- maxThreadsPerBlock – максимальное количество потоков на блок;
- maxThreadsDim[3] содержит максимальный размер каждого измерения блока;
- maxGridSize[3] содержит максимальный размер каждого измерения сетки;
- totalConstMem – общий объем константной памяти, доступной на устройстве в байтах;
- clockRate – тактовая частота в килогерцах;
- deviceOverlap равно 1, если устройство может одновременно копировать память между хостом и устройством во время выполнения ядра, или 0, если нет;
- multiProcessorCount – число мультипроцессоров на устройстве;

- kernelExecTimeoutEnabled равно 1, если существует ограничение времени выполнения для ядер, выполненных на устройстве, или 0, если нет.
- integrated равен 1, если устройство является интегрированным (материнская плата) GPU и 0, если оно является дискретным.
- canMapHostMemory равно 1, если устройство может отображать память хоста в адресное пространство CUDA для использования с cudaHostAlloc() / cudaHostGetDevicePointer(), или 0, если нет;
- concurrentKernels равно 1, если устройство поддерживает выполнение нескольких ядер в том же контексте одновременно, или 0, если нет.

Приведем пример использования функции cudaGetDeviceProperties:

```
cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, 0);//определение
параметров GPU с номером 0
printf("Device name : %s\n", deviceProp.name);
printf("Total global memory : %d MB\n", de-
viceProp.totalGlobalMem / 1024 / 1024);
printf("Shared memory per block : %d\n", de-
viceProp.sharedMemPerBlock);
printf("Registers per block : %d\n",   de-
viceProp.regsPerBlock);
printf("Warp size : %d\n", deviceProp.warpSize);
printf("Memory pitch : %d\n", deviceProp.memPitch);
printf("Max threads per block : %d\n", de-
viceProp.maxThreadsPerBlock);
printf("Max threads dimensions : x = %d, y = %d, z =
%d\n", deviceProp.maxThreadsDim[0], de-
viceProp.maxThreadsDim[1], de-
viceProp.maxThreadsDim[2]);
printf("Max grid size: x = %d, y = %d, z = %d\n", de-
viceProp.maxGridSize[0], deviceProp.maxGridSize[1],
deviceProp.maxGridSize[2]);
printf("Clock rate: %d\n", deviceProp.clockRate);
```

```
    printf("Total constant memory: %d\n", de-
    viceProp.totalConstMem);
    printf("Compute capability: %d.%d\n", de-
    viceProp.major, deviceProp.minor);
    printf("Texture alignment: %d\n", de-
    viceProp.textureAlignment);
    printf("Device overlap: %d\n", de-
    viceProp.deviceOverlap);
    printf("Multiprocessor count: %d\n", de-
    viceProp.multiProcessorCount);
    printf("Kernel execution timeout enabled: %s\n", de-
    viceProp.kernelExecTimeoutEnabled ? "true" :
    "false");
    scanf("");
```

Посмотрим, что выдала программа:

```
Device name : GeForce 940M
Total global memory : 2048 MB
Shared memory per block : 49152
Registers per block : 65536
Warp size : 32
Memory pitch : 2147483647
Max threads per block : 1024
Max threads dimensions : x = 1024, y = 1024, z = 64
Max grid size: x = 2147483647, y = 65535, z = 65535
Clock rate: 1176000
Total constant memory: 65536
Compute capability: 5.0
Texture alignment: 512
Device overlap: 1
Multiprocessor count: 3
Kernel execution timeout enabled: true
```

Также важным моментом является и определение версии видеокарты. Следующий код выводит производительность видеокарты

```
cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, 0);
printf("Device has Compute Capability %d.%d\n", de-
viceProp.major, deviceProp.minor);
```

Контрольные вопросы

1. Какие основные части содержит CUDA-программа?
2. Что такое ядро? Опишите все возможные спецификаторы функций и параметры ядра.
3. Как организована иерархия нитей в CUDA?
4. Какие типы синхронизации потоков существуют?
5. Что такое варп?

Лабораторная работа

1. Определите характеристики используемой видеокарты: число SM, объем памяти, частоту работы видеопроцессоров и памяти, ширину полосы пропускания памяти. Сделайте выводы о производительности видеокарты.

Глава 2. Расширение языка CUDA C

В этой главе мы узнаем, какие расширения получил язык программирования C/C++ для написания кода программ, реализуемых на GPU, и напишем свою первую программу.

Для программирования на CUDA предусмотрены специальные расширения языка C/C++. Они состоят из:

- спецификаторов функций, определяющих, откуда происходит вызов функций и где они исполняются;
- спецификаторов переменных, задающих тип памяти, выделяемой под эти переменные;
- встроенных векторных типов;
- встроенных переменных для задания размера сетки и блока, а также для индексации нитей внутри блока и блоков внутри сетки;
- специальных директив вызова ядра, задающих иерархию нитей.

Далее подробно рассмотрим каждое из этих расширений, но сначала давайте напишем свою первую программу на CUDA.

Первая программа

В любом учебном пособии первая программа, которую предлагают вам написать – это программа, выводящая на консоль «Hello, world!». Данная программа на языке программирования C++ выглядит следующим образом:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>

__global__ void HelloWorld()
{
  printf("Hello world, %d, %d\n", blockIdx.x,
threadIdx.x);
}
```

```
int main()
{
  HelloWorld << <1, 1 >> >();
// ожидаем нажатия любой клавиши
  getchar();
  return 0;
}
```

Первые две строки в программе (два `#include`) не обязательны. В среде Visual Studio они подключаются по умолчанию.

В отличие от файлов с программами на языке программирования C++, имеющих расширения .cpp, файлы с программами, которые используют технологию CUDA, должны иметь расширение .cu.

Спецификаторы функций и переменных

В расширении языка C/C++ используют три спецификатора функций, которые определяют, откуда функции вызываются и где выполняются (табл. 3).

Таблица 3

Спецификаторы функций в CUDA

Спецификатор	Функция выполняется на	Функция вызывается из
__global__	device (GPU)	host (CPU)
__device__	device (GPU)	device (CPU)
__host__	host (CPU)	host (CPU)

Спецификатор __global__ определяет ядро, и соответствующая функция должна возвращать тип void. При вызове ядра обязательно указывается конфигурация вызова.

На функции, исполняемые на устройстве (GPU) (спецификаторы __global__ и __device__) , накладываются следующие ограничения:

- не поддерживается рекурсия;

- не поддерживаются static переменные внутри функций;
- не поддерживается переменное число аргументов.

Спецификаторы __host__ и __device__ можно использовать совместно, это означает, что данная функция может выполняться как на устройстве, так и на хосте. Спецификаторы __global__ и __host__ не могут быть использованы совместно.

Адрес функции, помеченной спецификатором __device__, нельзя использовать в качестве указателя на функцию, а __global__ – можно.

Для размещения переменных в памяти устройства (GPU) используются следующие спецификаторы: __device__, __shared__, __constant__. Их основные свойства указаны в табл. 4.

Таблица 4

Спецификаторы переменных в CUDA

Спецификатор	Память GPU	Область видимости	Время жизни
__device__	глобальная	сетка (grid)	программа
__shared__	разделяемая	блок (block)	блок
__constant__	константная	сетка (grid)	программа

На использование введенных переменных накладывается ряд ограничений:

- соответствующие переменные могут быть объявлены только в пределах одного файла, то есть они не могут быть помечены квалификатором extern;
- для записи в переменную, помеченную как __constant__, должны использоваться специальные CUDA функции;
- переменная __shared__ не может быть проинициализирована при объявлении;
- указанные спецификаторы не могут применяться к полям структуры.

Встроенные типы данных и встроенные переменные

В CUDA добавлены 1/2/3/4-мерные векторы, образованные из базовых типов языка C:

```
    char1, uchar1, char2, uchar2, char3, uchar3, char4, uchar4;
    short1, ushort1, short2, ushort2, short3, ushort3, short4, ushort4;
    int1, uint1, int2, uint2, int3, uint3, int4, uint4;
    long1, ulong1, long2, ulong2, long3, ulong3, long4, ulong4;
    float1, float2, float3, float4;
    double1, double2.
```

Обращение к компонентам вектора осуществляется по названиям измерений: x, y, z и w, например,

```
    uint4 vec;
    int x = vec.x;
```

Для этого типа данных не предусмотрены привычные векторные операции, например, нельзя сложить два таких вектора с помощью оператора «+», необходимо использовать покомпонентное сложение. Для задания значений таких переменных предусмотрен оператор make_<typename>, например,

```
    int2 vec = make_int2(2,5); // создаем вектор с координатами (2,5)
```

Для задания размерности в CUDA добавлен специальный тип dim3. Он основан на типе данных uint3, но дополнительно снабжен конструктором, который заполняет все не указанные компоненты вектора единицами:

```
    dim3 block (16,16); // создали трехмерный вектор с компонентами (16,16,1)
```

Указанный тип данных удобно использовать при задании сетки для запуска основных вычислительных ядер.

Для того, чтобы ядро могло однозначно определить номер нити и, соответственно, номер элемента массива данных для обработки в CUDA предусмотрены встроенные переменные *threadIdx* и *blockIdx* типа dim3. Каждая из этих переменных является трехмерным целочисленным вектором. Размеры сетки (grid) могут быть получены ядром через встроенные переменные *blockDim* и *gridDim* также типа dim3.

Рассмотрим отображение локального номера нити и локального номера блока в глобальный номер нити. Схематично это отображение представлено на рис. 6.

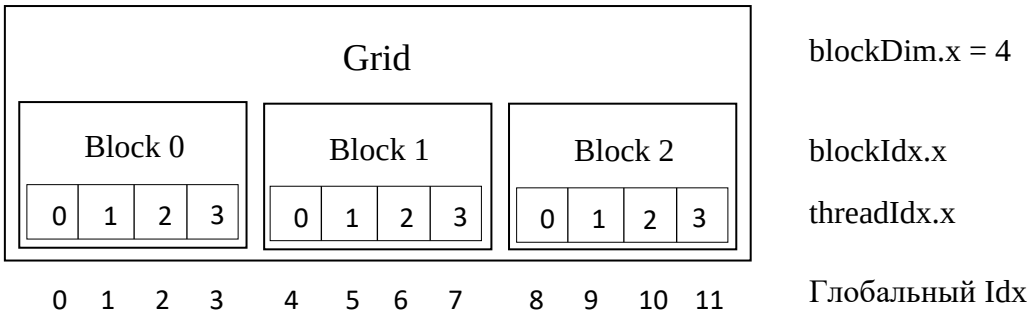


Рис. 6. Отображение локальных индексов нитей в глобальные

Зная номер нити внутри блока и номер блока внутри сетки, а также размерность сетки, можно вычислить глобальный индекс нити:

```
Idx = blockIdx.x * blockDim.x + threadIdx.x
```

Для получения глобального индекса нити в случае двумерных и трехмерных сеток сначала рекомендуется найти индекс нити относительно каждого из измерений:

```
Idx = blockIdx.x * blockDim.x + threadIdx.x
Idy = blockIdx.y * blockDim.y + threadIdx.y
```

Для двумерного массива данных размера (DimX, DimY) глобальный индекс нити вычисляется по формуле (см. рис.7):

```
Idx = Idx + Idy * DimX
```

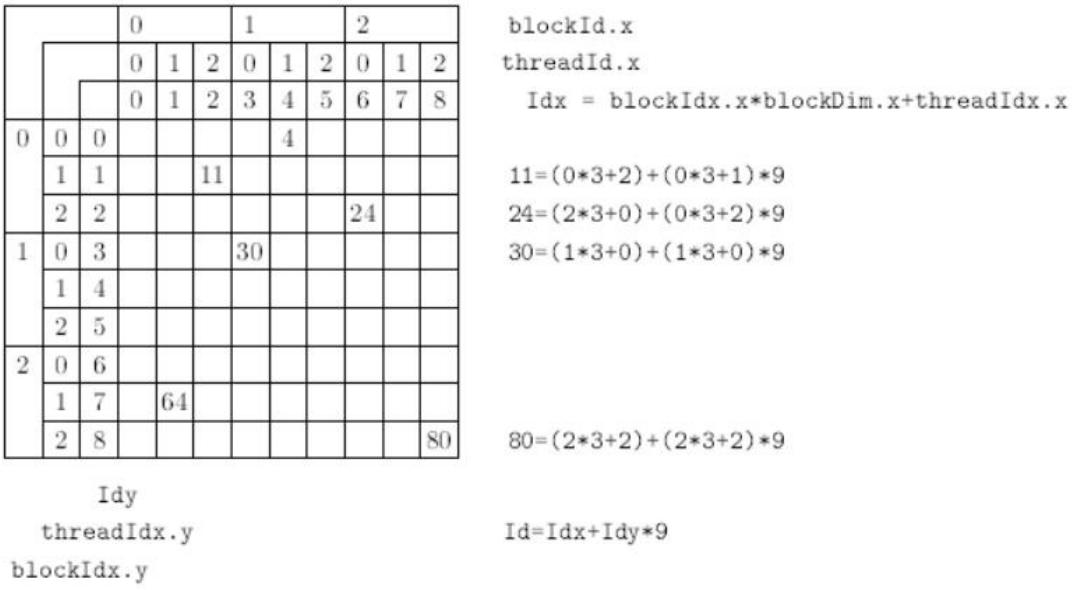


Рис. 7. Вычисление глобального номера нити для двумерной сетки

Максимальное число нитей в одном блоке ограничено, так как все они располагаются на одном потоковом процессоре и вынуждены делить между собой ограниченное количество ресурсов. Как правило, число нитей в блоке может достигать 512 или 1024 в зависимости от архитектуры устройства. Однако вычислительное ядро может быть запущено одновременно на большом количестве блоков, поэтому общее количество нитей равно произведению числа нитей в одном блоке на количество блоков.

Директивы вызова ядра

Как мы уже видели, для запуска ядра на GPU в простейшем случае используется конструкция вида:

```
kernel<<<GridDim, BlockDim>>>(args);
```

Здесь *kernel* – название соответствующей __global__ функции ядра, а *GridDim* и *BlockDim* – переменные типа dim3, задающие размерность сетки и блоков внутри сетки.

Размерность сетки (в блоках) и размерность блоков (в нитях) можно задать двумя способами:

```
dim3 grid, block;
grid.x = 2; grid.y = 4;
```

```
block.x = 8; block.y = 16;
```

или

```
dim3 grid(2, 4);
dim3 block(8,16);
```

Через `args` обозначены аргументы функции ядра, их может быть несколько. Рассмотрим следующий пример:

```
kernel<<<dim3(n/256), dim3(16,16)>>>(n, a);
```

В этом примере происходит вызов функции ядра kernel параллельно на n нитях (n целочисленно делится на 256), потому что количество нитей в одном блоке равно 16*16=256 (двумерный блок), а количество блоков в сетке равно n/256 (одномерная сетка блоков), таким образом, общее количество нитей равно n. В качестве аргументов функции передаются два параметра – a и n.

В примере ниже приведен фрагмент кода программы, увеличивающий все элементы массива на заданное число b, в реализации для CPU и GPU:

CPU:

```
float *data;
float b;
// … выделение памяти под указатель и инициализация значений
for(int i=0; i<n; i++)
  data[i] = data[i] + b;
```

GPU:

```
__global__ void kernel(float *data, float b){
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  data[i] = data[i] + b;
}
```

Контрольные вопросы

1. Перчислите расширения языка CUDA C.
2. Как можно вычислить глобальный номер нити для трехмерного массива размера (DimX, DimY, DimZ)?
3. Какие параметры задаются при вызове ядра?
4. Назовите спецификаторы переменных и функций в CUDA.

Лабораторная работа

1. Напишите функцию «Hello World» с различными возможными спецификаторами.
2. Реализуйте ядро, увеличивающее элементы двумерного массива на два.

Глава 3. Особенности написания программ на CUDA

Теперь, когда мы познакомились с базовыми понятиями CUDA и написали свою первую программу, перейдем к изучению основ работы с памятью на устройстве, узнаем, каким образом осуществляется взаимодействие CPU и GPU, как отслеживаются ошибки, которые неизбежно возникают при написании кода и, наконец, узнаем о способах синхронизации нитей, предусмотренных в CUDA. Также напишем еще одну типичную программу для GPU – сложение двух векторов.

Работа с памятью

Память на устройстве выделяется и освобождается CPU с помощью следующих функций (табл. 5).

Таблица 5

CUDA функции для выделения, копирования и освобождения памяти на GPU

Назначение функции	**Синтаксис функции**
Выделение памяти	cudaMalloc(void **devPtr, size_t size);
Освобождение памяти	cudaFree(void **devPtr);
Копирование данных	cudaMemcpy(void *dst, const void *src, size_t size, enum cudaMemcpyKind kind) В качестве параметра kind, задающего направление копирования, могут выступать: cudamemcpyHostToDevice (с хоста на устройство), cudamemcpyDeviceToHost (с устройства на хост), cudamemcpyDeviceToDevice (с устройства на устройство), cudamemcpyHostToHost (с хоста на хост).

Простой пример работы функций выделения, копирования и освобождения памяти приведен ниже:

```
    float *h_a, *h_b; // host
    float *d_a, *d_b; // device
    int n = 1024;
    int data_sz = n*sizeof(float);

    //Выделение памяти на CPU:
    h_a = (float *)malloc(data_sz);
    h_b = (float *)malloc(data_sz);

    //Выделение памяти на GPU:
    cudaMalloc((void **) &d_a, data_sz);
    cudaMalloc((void **) &d_b, data_sz);

    //Инициализация массива на CPU:
    for (int i=0; i<n; i++)
    h_a[i] = 10.f + i;

    //Копирование массива a на GPU:
    cudaMemcpy(d_a, h_a, data_sz, cudaMemcpyHostToDe-
vice);

    //Копирование массива b на GPU:
    cudaMemcpy(d_b, d_a, data_sz, cudaMemcpyDeviceToDe-
vice);

    //Копирование массива b на CPU:
    cudaMemcpy(h_b, d_b, data_sz,
cudaMemcpyDeviceToHost);

    //Сравнение результатов:
    for(int i=0; i<n; i++)
    assert(h_a[i] == h_b[i]);

    //Освобождение памяти на CPU и GPU:
    free(h_a);
    free(h_b);
    cudaFree(d_a);
    cudaFree(d_b);
```

Сложение двух векторов

Рассмотрим пример сложения двух векторов на GPU:

```
__global__ void sumKernel(float * a, float * b, float
* c)
{
// глобальный индекс нити
int i = threadIdx.x + blockIdx.x * blockDim.x;
c[i] = a[i] + b[i];
}
void sum(float * a, float * b, float * c, int n)
{
int data_sz = n * sizeof(float);
float * d_a;
float * d_b;
float * d_c;

// выделение памяти в GPU
cudaMalloc((void**)&d_a,data_sz);
cudaMalloc((void**)&d_b,data_sz);
cudaMalloc((void**)&d_c,data_sz);

//копирование данных из памяти CPU в память GPU
cudaMemcpy(d_a,a,data_sz,cudaMemcpyHostToDevice);
cudaMemcpy(d_b,b,data_sz,cudaMemcpyHostToDevice);

//вызов ядра для обработки данных
sumKernel <<<dim3(512),dim3(n/512)>>>(d_a,d_b,d_c);

//копирование результатов в память CPU
cudaMemcpy(c,d_c,data_sz,cudaMemcpyDeviceToHost);

//освобождение памяти
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);
}
```

Первая функция sumKernel() является ядром, выполняющим сложение векторов. Спецификатор __global__ указывает на то, что функция вызывается из CPU и выполняется на GPU. Ядро при помощи встроенных пе-

ременных вычисляет глобальный индекс нити, для которого выполняется сложение, и суммирует соответствующие элементы. Функция sum() выделяет память на GPU, копирует входные данные из памяти на CPU в память GPU, вызывает ядро sumKernel(), копирует результат обратно в память CPU и освобождает выделенную память.

Типичная структура CUDA программы

В приведенном выше примере хорошо показаны этапы вызова CUDA функций. Как правило, типичная структура CUDA программы имеет следующий вид:

1. Объявление глобальных переменных;
2. Описание прототипов функций:
   - __global__ void kernel();
3. Функция main():
   - Размещение памяти на устройстве cudaMalloc();
   - Передача данных с процессора на устройство cudaMemcpy();
   - Определение параметров запуска ядра;
   - Вызов ядра kernel<<<block, thread>>>();
   - Передача результатов с устройства на процессор cudaMemcpy();
   - Дополнительно: сравнение результатов расчета на процессоре и устройстве;
4. Ядро __global__ void kernel():
   - Объявление переменных local, shared;
   - Синхронизация __syncthreads().

Обработка ошибок

После выполнения функций на GPU, они возвращают значения типа cudaError_t. Функции, успешно отработавшие на девайсе, возвращают значение cudaSuccess, в противном случае возвращается код ошибки. Таблица?

Назначение функции	Синтаксис функции
Возвращает описание ошибки в виде строки	cudaGetErrorString(cudaError_t error);
Возвращает код последней ошибки	cudaGetLastError();

Основные выводы ответа при обработке ошибок:

1) cudaSuccess – при удачном выполнении;

2) cudaErrorMemoryAllocation – при ошибке выделения памяти;

3) cudaErrorInvalidValue – неверные параметры аргумента (например, размер копирования отрицателен);

4) cudaErrorInvalidDevicePointer – неверный указатель памяти в видеокарте;

5) cudaErrorInvalidMemcpyDirection – неверное направление (например, перепутан источник и место назначения копирования);

6) cudaErrorInitializationError – ошибка инициализации;

7) cudaErrorPriorLaunchFailure – ошибка при предыдущем асинхронном запуске функции;

8) cudaErrorInvalidResourceHandle – неверный обработчик события.

Приведем пример использования функции cudaGetLastError().

```
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) printf("%s ", cudaGetError-
String(err));
```

Здесь проверяется, есть ли ошибка у последней команды, выполненной на девайсе. Если ошибка есть, то выводится ее описание.

Замер времени работы части кода программы

Одним из важных параметров при написании программы является время, затраченное на ее выполнение. Для замера времени выполнения операций на CPU есть множество возможностей. Для того, чтобы замерить время, которое было затрачено на выполнение вычислений на GPU, можно воспользоваться

механизмом событий (CUDA events). Событие – это объект типа cudaEvent_t, используемый для обозначения «точки» среды вызовов CUDA.

Приведем пример фрагмента кода, замеряющего время выполнения ядра на GPU:

```
// инициализируем события
cudaEvent_t start, stop;
float elapsedTime;

// создаем события
cudaEventCreate(&start);
cudaEventCtreate(&stop);

// запись события
cudaEventRecord(start, 0);

// вызов ядра
cudaEventRecord(stop,0);

// ожидание завершения работы ядра
cudaEventSynchronize(stop);
cudaEventElapsedTime(&elapsedTime, start, stop);

// вывод информации
printf("Time spent executing by the GPU: %.2f
millseconds\n", elapsedTime);

// уничтожение события
cudaEventDestroy(start);
cudaEventDestroy(stop);
```

Синхронизация

Для синхронизации текущей нити на CPU с GPU используется функция cudaThreadSynchronize(), которая дожидается завершения выполнения всех операций CUDA, ранее вызванных с данной нити CPU. Функция CPU cudaDeviceSynchronize() блокирует выполнение кода CPU, пока все GPU ядра

не выполнены. Для синхронизации всех нитей в одном и том же блоке используется функция __syncthreads(), которая блокирует вызывающие нити до тех пор, пока все нити блока не войдут в эту функцию. При помощи этой функции можно организовать барьеры внутри ядра, гарантирующие, что если хотя бы одна нить прошла такой барьер, то не осталось ни одной за барьером (не прошедшей его). Такая синхронизация называется барьерной (рис. 8).

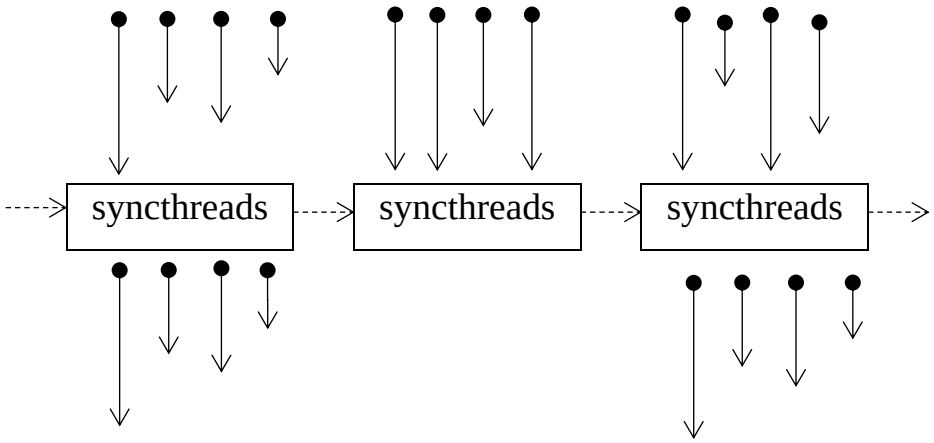


Рис. 8. Барьерная синхронизация

Нежелательно размещать эту команду внутри условного оператора **if**, а также следует обеспечивать безусловный вызов этой функции всеми потоками блока.

Выпишем остальные «барьерные команды»:

- **__threadfence_block()** заставляет ждать вызвавший её поток, пока все совершенные операции доступа к разделяемой и глобальной памяти завершатся и станут видны потокам этого блока.
- **__threadfence()** заставляет ждать вызвавший её поток, пока все совершенные операции доступа к разделяемой памяти станут видны потокам этого блока, а операции с глобальной памятью – всем потокам на девайсе.
- **__threadfence_system()** подобна __threadfence(), но включает синхронизацию с потоками на CPU («хосте»), при использовании page-locked памяти.

Контрольные вопросы

1. Перечислите виды синхронизации.
2. Какие основные ошибки возникают при исполнении кода на CUDA?
3. Опишите типичную структуру CUDA программы.

Лабораторная работа

1. Выделите память под двумерный и трехмерный массивы, заполните его значениями и скопируйте на девайс.

2. Постройте зависимость скорости копирования данных от объема для одномерного массива с CPU на GPU и наоборот.

Домашняя работа

1. Измерьте время выполнения кода функции сложения двух векторов с помощью функций C и cudaEvent. Объясните разницу.

Глава 4. Атомарные операции в CUDA

CUDA для GPU, начиная с производительности 1.1 и выше, поддерживает атомарные операции над глобальной и разделяемой памятью. Атомарность заключается в том, что гарантируется корректность выполнения операции в случае, когда много нитей пытаются ее выполнить одновременно.

Атомарные арифметические операции

Наиболее часто используемыми атомарными арифметическими операциями являются atomicAdd и atomicSub, служащие для увеличения или уменьшения величины на заданное значение. При этом функция возвращает старое значение. Нужно обратить внимание, что atomicAdd поддерживает операции над 64-битовыми величинами, но только расположенными в глобальной памяти.

```
int atomicAdd( int *addr, int value );
unsigned int atomicAdd( unsigned int *addr, unsigned
int value );

unsigned  long long atomicAdd(unsigned  long long
*addr, unsigned  long long value );

int atomicSub( int *addr, int value );

unsigned  int atomicSub(unsigned  int *addr, unsigned
int value );
```

Операция atomicExch осуществляет атомарный обмен значениями – передаваемое значение записывается по указанному адресу, а предыдущее значение возвращается. При этом подобный обмен происходит как одна транзакция, то есть ни одна нить не может «вклиниться» между шагами этого обмена. Операции над 64-битовыми значениями поддерживаются только для глобальной памяти.

```
int atomicExch( int *addr, int value );

unsigned int atomicExch(unsigned int *addr, unsigned
int value );

unsigned long long atomicExch(unsigned long long
*addr, unsigned long long value );

float atomicExch( float *addr, float value );
```

Следующие две операции сравнивают значение по адресу с переданным значением, записывают минимум/максимум из этих двух значений по заданному адресу и возвращают предыдущее значение, находившееся по адресу. Все эти шаги выполняются атомарно, как одна транзакция.

```
int atomicMin( int *addr, int value );

unsigned int atomicMin(unsigned int *addr, unsigned
int value );

int atomicMax( int *addr, int value );

unsigned int atomicMax(unsigned int *addr, unsigned
int value );
```

Операция atomicInc читает слово по заданному адресу и сравнивает его с переданным значением. Если прочитанное слово больше, то по адресу записывается нуль, иначе значение по адресу увеличивается на единицу. Возвращается старое значение.

```
unsigned int atomicInc(unsigned int *addr, unsigned
int value );
```

Операция atomicDec читает слово по переданному адресу. Если прочитанное значение равно нулю или больше переданного значения, то записывает

по адресу переданное значение, иначе уменьшает значение по адресу на единицу. Возвращается старое значение.

```
unsigned int atomicDec(unsigned int *addr, unsigned
int value );
```

Следующая функция (CAS – Compare And Swap) читает старое 32- или 64-битовое значение по переданному адресу и сравнивает его с параметром compare. В случае совпадения по переданному адресу записывается значение параметра value, иначе значение по адресу не изменяется. Возвращается всегда старое прочитанное значение.

```
int atomicCAS( int *addr, int compare, int value );

unsigned int atomicCAS(unsigned int *addr, unsigned
int compare, unsigned int value );

unsigned long long atomicCAS(unsigned int *addr, un-
signed long long compare, unsigned long long value );
```

Атомарные побитовые операции

Побитовые атомарные операции читают слово по заданному адресу, применяют к нему заданную побитовую операцию с заданным параметром и записывают результат обратно. Возвращается всегда старое значение, находившееся по переданному адресу до начала операции.

```
int atomicAnd( int *addr, int value );

unsigned int atomicAnd(unsigned int *addr, unsigned
int value );

int atomicOr( int *addr, int value );
```

```
unsigned int atomicOr(unsigned int *addr, unsigned
int value );

int atomicXor( int *addr, int value );

unsigned int atomicXor(unsigned int *addr, unsigned
int value );
```

Дополнительные возможности Compute Capability 6.x

Начиная с производительности 6.x, существует ряд нововведений. В частности, представлен новый тип атомарных операций. Например, atomicAdd_system гарантирует, что инструкция является атомарной относительно других CPU и GPU в системе. Функция atomicAdd_block подразумевает, что инструкция является атомарной только с учетом атомистики от других потоков в одном блоке потока. В следующем примере как CPU, так и GPU могут атомарно обновлять целочисленное значение по адресу addr:

```
__global__ void add10(int *addr) {
    // only available on devices with compute
    // capability 6.x
    atomicAdd_system(addr, 10);
}

void foo()
{
    int *addr;
    cudaMallocManaged(&addr, 4);
    *addr = 0;
    add10<<<2, 5>>>(addr);
    // CPU atomic operation
    _sync_fetch_and_add(addr, 10);
}
```

Также в версиях производительности 6.x добавлено сложение 64-битных чисел.

Пример использования атомарных операций

Пусть задана двумерная сетка нитей, и требуется сосчитать количество нитей, индексы которых удовлетворяют некоторому условию. Эту задачу можно реализовать следующим способом: каждая нить вызывает некую булеву функцию, и если результат вызова этой функции равен true, добавляет единицу некой глобальной переменной res.

Приведем ядро такой программы.

```
__global__ void Kernel(int * res) {

  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int j = blockIdx.y * blockDim.y + threadIdx.y;

  if ( Condition(i, j) )// проверка условия
     {
    atomicAdd(res, 1);
  }
}
```

Здесь `Condition(i, j)` некоторое условие, например, `i>=j`.

Контрольные вопросы

1. Что такое атомарная операция? В чем ее ключевое отличие от обычных операций?

2. Какие атомарные операции вы знаете?

Лабораторная работа

1. Вычислите приблизительное значение числа π методом Монте-Карло. Постройте равномерную сетку нитей в квадрате [0, 1]X[0, 1] и посчитайте количество нитей, которые попали в круг с центром в нуле и с радиусом, равным единице. Для вычисления случайного числа напишите __device__ функцию.

Домашняя работа

1. Вызов атомарной функции в целом замедляет работу программы. Например, если очень много нитей должны добавить единицу одной и той же переменной, то они встанут в очередь и будут простаивать, пока все по очереди не выполнят эту операцию. Одним из подходов для решения этой проблемы является уменьшение количества нитей, вызывающих атомарную операцию. Учитывая вышесказанное, подумайте, как можно уменьшить количество нитей, вызывающих атомарную операцию в методе Монте-Карло. Оптимизируйте таким образом программу и посмотрите, насколько уменьшается время ее работы. Также посмотрите, как меняется точность вычисления числа π.

Глава 5. Работа с векторами. Математические функции

В языке Си (в частности, и в диалекте CUDA C) векторы обычно представляются массивами, как правило, динамическими (память выделяется с помощью функций семейства malloc() и освобождается вызовом free(), аналоги в CUDA см. ниже).

В функции же передается не сам массив целиком, а лишь *указатель* на его начало (переменная фактически целочисленного типа, размер которого зависит от архитектуры (как правило, 32 или 64 разряда), содержащая адрес начала массива). Зная размер одного элемента, легко вычислить адрес любого элемента по его смещению.

Сложение векторов

Давайте вспомним программу, которая генерирует два вектора целых чисел, копирует их в память GPU, выполняет сложение на GPU, результат копирует обратно в память CPU. GPU обычно не работает напрямую с памятью CPU, хотя это технически возможно. Для начала необходимо выделить память под данные на device, для этого используется фукция cudaMalloc(), после этого копируем данные из памяти host в память device с помощью cudaMemcpy(). Далее для каждого элемента вектора запускается отдельный процесс сложения, т. е. в данном случае – 128 нитей. На CPU это могло быть накладно, но GPU как раз рассчитан на работу с большим количеством потоков, и всё происходит достаточно быстро. После того, как операция завершена с помощью cudaMemcpy(), копируем результат обратно на host и освобождаем память device с помощью cudaFree(). В тексте ядра есть переменная threadIdx, это системная переменная CUDA, содержащая номер нити.

Приведем целиком программу сложения двух векторов с помощью технологии CUDA.

```
#define N 128 //количество элементов массива
// Ядро
__global__ void add( int *a, int *b, int *c )
```

```
{
    //вычисление индекса элемента
    int tid = threadIdx.x;
    //проверка на выход за пределы массива
    if(tid > N-1) return;
    //поэлементное сложение массивов
    c[tid] = a[tid] + b[tid];
}

int main() {
    // выделение памяти под массивы на CPU
    int host_a[N], host_b[N], host_c[N];
    // выделение памяти под массивы для копирования
    // на GPU
    int *dev_a, *dev_b, *dev_c;
    // заполнение массивов
    for (int i=0; i<N; i++)
    {
      host_a[i] = i * i;
        host_b[i] = - i;
    }
    // выделение памяти под массивы на GPU
    cudaMalloc( (void**)&dev_a, N * sizeof(int) );
    cudaMalloc( (void**)&dev_b, N * sizeof(int) );
    cudaMalloc( (void**)&dev_c, N * sizeof(int) );

    // копирование данных в память GPU
    cudaMemcpy( dev_a, host_a, N * sizeof(int),
    cudaMemcpyHostToDevice ) ;
    cudaMemcpy( dev_b, host_b, N * sizeof(int),
    cudaMemcpyHostToDevice ) ;

    // вызов ядра
    add<<<1, N>>>( dev_a, dev_b, dev_c );

    // получаем результат расчета
    cudaMemcpy( host_c, dev_c, N * sizeof(int),
    cudaMemcpyDeviceToHost ) ;
```

```
    // вывод результатов
    for (int i=0; i<N; i++)
    {
        printf( "%d + %d = %d\n", host_a[i],
        host_b[i], host_c[i] );
    }

    // освобождение памяти
    cudaFree( dev_a );
    cudaFree( dev_b );
    cudaFree( dev_c );
    return 0;
}
```

Вычисление математических функций

В CUDA существуют аналоги ряда математических функций, включенных в стандартную библиотеку языка Си (<math.h>). Так, например, прототип CUDA-версии синуса (sinf) имеет вид

```
__device__ float __sinf(float x);
```

Перечислим основные различия:

– функции CUDA вычисляются непосредственно на GPU;

– большинство функций работают только с float-аргументами (GPU, как правило, в основном работают с float, точности достаточно для основных задач, возлагаемых на GPU);

– имеют модификатор __device__, вследствие чего могут вызываться только с GPU (т.е. из kernel-функций);

Рассмотрим следующую задачу:

1. Выделить на GPU массив из n = 10^9 элементов типа float.

2. Инициализировать их в ядре следующим образом:

```
arr[i] = sin((i % 360) * pi / 180)
```

где `pi = 3.141592653`.

3. Скопировать массив в память CPU, посчитать ошибку

```
err = sum_i(abs(sin((i % 360) * pi / 180) - arr[i]))
/ n
```

4. Провести исследование при использовании массива типа double.

Основной частью данной задачи является написание ядра, в котором будет инициализироваться массив (и вызываться синус).

```
#define BASE_TYPE float
#define MAX_GRIDSIZE 1

__global__ void sinMass(BASE_TYPE *A, int arraySize)
{
    int index = (blockIdx.y * MAX_GRIDSIZE + blockIdx.x)
    * blockDim.x + threadIdx.x;
  if (index < arraySize)
        [index] = sin((BASE_TYPE)((index % 360) * M_PI /
        180));

    // другие варианты вычисления синуса
    // A[index] = sinf((index % 360) * M_PI / 180);
    // A[index] = __sinf((index % 360) * M_PI
    // / 180);
}
```

Приведем список встроенных математических функций для работы с типом float:

Функция	**Операция**
`__fadd_[rn,rz,ru,rd](x,y),`	x + y
`__fsub_[rn,rz,ru,rd](x,y),`	x – y
`__fmul_[rn,rz,ru,rd](x,y),`	x * y
`__fmaf_[rn,rz,ru,rd](x,y,z)`	x * y + z

`__frcp_[rn,rz,ru,rd](x)`	1 / x
`__fsqrt_[rn,rz,ru,rd](x)`	$\sqrt{x}$
`__frsqrt_rn(x)`	$1 / \sqrt{x}$
`__fdiv_[rn,rz,ru,rd](x,y)`	x / y
`__fdividef(x,y)`	x / y; при $y>2^{126}$ результат = 0
`__expf(x)`	e^x
`__exp10f(x)`	10^x
`__logf(x)`	ln x
`__log2f(x)`	$\log_2 x$
`__log10f(x)`	lg x
`__sinf(x)`	sin x
`__cosf(x)`	cos x
`__sincosf(x,sptr,cptr)`	*sptr = sin x, cptr = cos x
`__tanf(x)`	tg x
`__powf(x, y)`	x^y

Здесь `rn,rz,ru,rd` - способы округления, например, `__fadd_rn` округляет сумму к ближайшему, `__fadd_rz` – к нулю, `__fadd_ru` – до большего, `__fadd_rd` – до меньшего.

Список встроенных функций для работы с типом double несколько меньше:

Функция	**Операция**
`__dadd_[rn,rz,ru,rd](x,y),`	x + y
`__dsub_[rn,rz,ru,rd](x,y),`	x – y
`__dmul_[rn,rz,ru,rd](x,y),`	x * y
`__fma_[rn,rz,ru,rd](x,y,z)`	x * y + z
`__drcp_[rn,rz,ru,rd](x)`	1 / x
`__dsqrt_[rn,rz,ru,rd](x)`	$\sqrt{x}$
`__ddiv_[rn,rz,ru,rd](x,y)`	x / y

Контрольные вопросы

1. Как организовать обращение к элементу массива в теле ядра?
2. Какие основные этапы работы с векторами можно выделить?
3. Какие встроенные (быстрые) математические функции есть на GPU? В чем отличие их от обычных функций?

Лабораторная работа

1. Используя другие функции вместо синуса (__expf` (e^x), __exp10f` (10^x) и т. д.), рассчитайте относительную ошибку (отношение абсолютной ошибки к значению функции).

2. Напишите программу, реализующую скалярное произведение двух векторов.

Домашняя работа

1. Сравните скорость и точность вычислений с типами float и double для функций из лабораторной работы. Какой тип для каких задач лучше использовать и почему? Сделайте вывод.

2. Напишите программу, реализующую процесс ортогонализации Грама-Шмидта. При этом на хосте создается набор из N векторов вида a_1 = (1,1,1,1,…), a_2 = (0,1,1,1,…), a_3 = (0,0,1,1,…), … , a_N = (…,0,0,0,1). Далее эти вектора передаются на девайс, после чего производится ортогонализация. Алгоритм процесса ортогонализации (получение ортогональных векторов b_i по a_i) следующий:

$$b_1 = a_1, \qquad b_i = a_i - \sum_{k=1}^{i-1} \frac{(a_i, b_k)}{(b_k, b_k)} b_k, \quad i = 2..N.$$

.

Глава 6. Работа с матрицами

Перед тем, как начать работать с матрицами, скажем несколько слов о типе dim3 – одном из базовых типов CUDA. Он представляет собой не что иное, как вектор из 3-х целочисленных значений. Создается он следующим образом:

```
dim3 v1(512, 2, 4);
dim3 v2(512, 128);
dim3 v3(1024).
```

Если какие-то значения опущены (как в примере с v2 и v3), то они по умолчанию равны 1 (а не 0). То есть вышеприведенный код эквивалентен следующему:

```
dim3 v1(512, 2, 4);
dim3 v2(512, 128, 1);
dim3 v3(1024, 1, 1).
```

В CUDA есть несколько векторных типов для доступа к данным. Например, такой тип, как [u]int может быть 1-, 2-, 3-, 4-мерным вектором. Для создания переменной такого типа применяется функция специального вида, например:

```
int3 a = make_int3 ( 1, 2, 3);
```

Также существуют типы (u)char, (u)short, (u)long, float, которые могут быть 1-, 2-, 3-, 4-мерными векторами, а вот типы long long и double могут быть только 1- и 2-мерными.

На основе типа uint3, в CUDA существует специальный тип dim3, который имеет конструктор. Конструктор этого типа позволяет задавать не все компоненты 3-мерного вектора. Недостающие компоненты этого типа в конструкторе инициализируются единицами. Главное назначение типа dim3 – это задание параметров запуска ядра.

Внутри каждого вычислительного ядра содержатся встроенные переменные, которые используют вышеописанные типы. С помощью этих встроенных переменных можно оперировать отдельными нитями и различать их. Эти переменные были описаны выше, кроме переменной ***warpSize*** типа int – эта переменная содержит значение размера связки нитей (варп), которое на текущий момент всегда равно 32.

Используя встроенные переменные, можно задавать разные параметры запуска ядра и различать отдельные нити в блоках или сами блоки.

Создание матриц

Создадим на CPU матрицу A размером 10x10 и зададим элементы матрицы следующим образом:

$$A = \begin{pmatrix} 0 & 1 & 2 & ... & 9 \\ 10 & 11 & 12 & ... & 19 \\ 20 & 21 & 22 & ... & 29 \\ ... & ... & ... & ... & ... \\ 90 & 91 & 92 & ... & 99 \end{pmatrix}$$

Создадим на GPU матрицу B размером 10x10. Элементы матрицы B зададим такими же, как и у матрицы A. При этом не будем использовать копирование данных с хоста на девайс. Необходимо скопировать матрицу B с девайса на хост и сравнить на хосте две матрицы – A и B. Нужно добиться, чтобы обе матрицы совпадали. Попробуйте запускать программу с разными размерами grid и block.

Для решения этой задачи используем один блок с десятью нитями так, чтобы каждая нить создавала один элемент матрицы. Для удобства блок нитей мы сделаем двумерным. Таким образом, индекс элемента матрицы вычислим как threadIdx.y * n + threadIdx.x. В итоге код ядра будет выглядеть так:

```
__global__ void createMatrix(int *A, const int n)
```

```
{
    A[threadIdx.y * n + threadIdx.x] = 10 *
    threadIdx.y + threadIdx.x;
}
```

На CPU матрицу A будем создавать с помощью двойного цикла следующим образом:

```
size_t size = n * n * sizeof(int);
int *h_A = (int *)malloc(size);
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    h_A[j * n + i] = 10 * j + i;
```

Вызов ядра, описанного выше, осуществляется таким образом:

```
dim3 threadsPerBlock = dim3(10, 10);
dim3 blocksPerGrid = dim3(1);

createMatrix<<<blocksPerGrid, threadsPerBlock>>>
(d_B, n);
```

Проверку равенства двух матриц осуществляем следующим образом с помощью двойного цикла:

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
  if (h_A[j * n + i] != h_B[j * n + i]) {
        printf("h_A[%d] != h_B[%d]\n", j * n + i, j
        * n + i);
  }
```

Листинг кода.

```
// Сравнение двух матриц
#include "cuda_runtime.h"
```

```
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
// Ядро
__global__ void createMatrix(int *A, const int n)
{
  // Создание элементов матрицы на GPU
    A[threadIdx.y * n + threadIdx.x] = 10 *
    threadIdx.y + threadIdx.x;
}
int main()
{
  // кол-во строк и столбцов матрицы
    const int n = 10;

    // размер матрицы
    size_t size = n * n * sizeof(int);

    // выделяем память для матрицы на CPU
    int *h_A = (int *)malloc(size);

    // инициализируем матрицу
    for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
    h_A[j * n + i] = 10 * j + i;
    int *d_B = NULL;
    // выделяем память для матрицы на GPU
    cudaMalloc((void **)&d_B, size);

    // определение размеров сетки и блоков
    dim3 threadsPerBlock = dim3(10, 10);
    dim3 blocksPerGrid = dim3(1);

    // вызов ядра
    createMatrix<<<blocksPerGrid, threadsPerB-
    lock>>>(d_B, n);
```

```
    // выделяем память для матрицы B, чтобы
    // скопировать из GPU на CPU
    int *h_B = (int *)malloc(size);

    // копируем матрицу из GPU на CPU
    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceTo-
    Host);

    // проверяем совпадение матрицы A и матрицы B
    for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
            if (h_A[j * n + i] != h_B[j * n + i]) {
                printf("h_A[%d] != h_B[%d]\n", j * n
                + i, j * n + i);
            }

    // освобождаем память на GPU
    cudaFree(d_B);

    // освобождаем память на CPU
    free(h_A);
    free(h_B);
    return 0;
}
```

Транспонирование матрицы

Для решения этой задачи используем блоки размером 16x16. Каждая нить блока создает один элемент транспонированной матрицы. Код ядра будет выглядеть так:

```
__global__ void matrixTranspose (const BASE_TYPE *A,
BASE_TYPE *AT, int rows, int cols)
{
    // Индекс элемента в исходной матрице
```

```
    int iA = cols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x +
    threadIdx.x;
    // Индекс соответствующего элемента в
    // транспонированной матрице
    int iAT = rows * (blockDim.x * blockIdx.x +
    threadIdx.x) + blockDim.y * blockIdx.y +
    threadIdx.y;  AT[iAT] = A[iA];
}
```

Для удобства вычислений размеры исходной матрицы делаем кратными размеру блока. Для этого используем функцию toMultiple(int a, int b), которая возвращает число, кратное b и наиболее близкое к a. Матрицу инициализируем случайными числами.

```
// Инициализация матрицы
for (int i = 0; i < rows * cols; ++i)
{
    h_A[i] = rand()/(BASE_TYPE)RAND_MAX;
}
```

Вызов ядра, описанного выше, осуществляется таким образом:

```
dim3 threadsPerBlock = dim3(BLOCK_SIZE, BLOCK_SIZE);

dim3 blocksPerGrid = dim3(cols / BLOCK_SIZE, rows /
BLOCK_SIZE);

//вызов ядра
matrixTranspose<<<blocksPerGrid, threadsPerB-
lock>>>(d_A, d_AT, rows, cols);
```

Осуществляем проверку правильности работы ядра.

```
for (int i = 0; i < rows; i++)
```

```
  for (int j = 0; j < cols; j++)
        if (h_A[i * cols + j] != h_AT[j * rows + i])
        {
            fprintf(stderr, "Result verification
            failed at element [%d, %d]!\n", i, j);
            exit(EXIT_FAILURE);
    }
```

Листинг кода:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

// размер блока
#define BLOCK_SIZE 16

// тип, который будут иметь элементы матриц
#define BASE_TYPE float

// Ядро
// Функция транспонирования матрицы
__global__ void matrixTranspose(const BASE_TYPE *A,
BASE_TYPE *AT, int rows, int cols)
{
    // Индекс элемента в исходной матрице
    int iA = cols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x +
    threadIdx.x;

    // Индекс соответствующего элемента в
    // транспонированной матрице
    int iAT = rows * (blockDim.x * blockIdx.x +
    threadIdx.x) + blockDim.y * blockIdx.y +
    threadIdx.y;
    AT[iAT] = A[iA];
}
```

```
// Функция вычисления числа, которое больше числа a
// и кратное числу b
int toMultiple(int a, int b)
{
    int mod = a % b;
    if (mod != 0)
    {
        mod = b - mod;
        return a + mod;
    }
    return a;
}

int main()
{
    // Объекты событий
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

  // Количество строк и столбцов матрицы
    int rows = 1000;
    int cols = 2000;

  // Меняем количество строк и столбцов матрицы
    // на число, кратное размеру блока (16)
    rows = toMultiple(rows, BLOCK_SIZE);
    printf("rows = %d\n", rows);
    cols = toMultiple(cols, BLOCK_SIZE);
    printf("cols = %d\n", cols);
    size_t size = rows * cols * sizeof(BASE_TYPE);

  // Выделение памяти под матрицы на хосте
  // Исходная матрица
    BASE_TYPE *h_A = (BASE_TYPE *)malloc(size);
  // Транспонированная матрица
    BASE_TYPE *h_AT = (BASE_TYPE *)malloc(size);
```

```
// Инициализация матрицы
  for (int i = 0; i < rows * cols; ++i)
  {
      h_A[i] = rand()/(BASE_TYPE)RAND_MAX;
  }
// Выделение глобальной памяти на девайсе
  // для исходной матрицы
  BASE_TYPE *d_A = NULL;
  cudaMalloc((void **)&d_A, size);

  // Выделение глобальной памяти на девайсе для
  // транспонированной матрицы
  BASE_TYPE *d_AT = NULL;
  cudaMalloc((void **)&d_AT, size);

  // Копируем матрицу из CPU на GPU
  cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDe-
  vice);

  // Определяем размер блока и сетки
  dim3 threadsPerBlock = dim3(BLOCK_SIZE,
  BLOCK_SIZE);
  dim3 blocksPerGrid = dim3(cols / BLOCK_SIZE,
  rows / BLOCK_SIZE);

  // Начать отсчета времени
  cudaEventRecord( start, 0);

  // Запуск ядра
  matrixTranspose<<<blocksPerGrid, threadsPerB-
  lock>>>(d_A, d_AT, rows, cols);

  // Окончание работы ядра, остановка времени
  cudaEventRecord( stop, 0);
  cudaEventSynchronize( stop );
  float KernelTime;
  cudaEventElapsedTime( &KernelTime, start, stop);
```

```
    printf("KernelTime: %.2f milliseconds\n", Ker-
    nelTime);

    // Копируем матрицу из GPU на CPU
    cudaMemcpy(h_AT, d_AT, size, cudaMemcpyDeviceTo-
    Host);

    // Проверка правильности работы ядра
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
        {
            if (h_A[i * cols + j] != h_AT[j * rows +
            i])
            fprintf(stderr, "Result verification
            failed at element [%d, %d]!\n", i, j);
            exit(EXIT_FAILURE);
        }
    printf("Test PASSED\n");

    // Освобождаем память на GPU
    cudaFree(d_A);
    // Освобождаем память на GPU
    cudaFree(d_AT);
  // Освобождаем память на CPU
    free(h_A);
    free(h_AT);
      // Удаляем объекты событий
    cudaEventDestroy( start );
  cudaEventDestroy( stop );
  printf("Done\n");
  return 0;
}
```

Сложение матриц

Поскольку сложение матриц выполняется поэлементно (еще проще, чем умножение), то все сказанное в следующем разделе (по поводу параллельного

вычисления и представления массивов в памяти) справедливо и для сложения матриц.

Для решения задачи используем функцию __global__ void matrixAdd.

```
// Ядро
__global__ void matrixAdd(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int cols)
{
    // Вычисление индекса элемента матрицы на GPU
    int ind = cols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

    C[ind] = A[ind] + B[ind];
}

// Функция вычисление числа, которое больше
// числа a и кратное числу b
int toMultiple(int a, int b)
{
     int mod = a % b;
     if (mod != 0)
     {
     mod = b - mod;
     return a + mod;
     }
     return a;
}
```

Умножение матриц

Пусть даны матрицы $A = (a_{ij})$ и $B = (b_{ij})$ размерами $m \times n$ и $n \times p$ соответственно. Тогда произведение $C = (c_{ij}) = AB$ размерностью $m \times p$, как известно, определяется по формуле

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Элементы матрицы-произведения зависят только от элементов исходных матриц, поэтому их вычисление может выполняться параллельно, без дополнительных «ухищрений». Поэтому рассмотрим лишь вопрос о вариантах представления матрицы в памяти – в виде двухмерного (например, float** a, затем обращение вида a[i][j]) или одномерного массива (float* a и затем обращение вида a[i * N + j], где N – длина строки (количество столбцов)).

Рассмотрим вопрос о том, как работает операция индексирования:

1) в случае с одномерным массивом:

a) вычисляется выражение i * N + j (в современных CPU это выполняется за одну операцию, так называемую fused multiply-add);

b) полученное число (смещение) умножается на размер типа и складывается с базовым адресом (адресом начала массива);

c) как правило, переменные i, j и базовый адрес будут храниться в регистрах CPU, поэтому вышеприведенные операции выполняются быстро.

2) в случае с двухмерным массивом:

a) значение i умножается на размер указателя и складывается с базовым адресом, вычисляется адрес a[i] (начала i-й строки);

b) Значение j умножается на размер элемента и складывается с адресом a[i], вычисляется адрес a[i][j];

c) Адрес a[i], очевидно, не хранится в регистре, поэтому операция с ним будет медленнее.

С другой стороны, одномерный массив требует большого *последовательного* участка памяти.

Для решения задачи опишем функцию __global__ void matrixMult.

```
__global__ void matrixMult(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int Acols, int Bcols)
{
    int i0 = Acols * (blockDim.y * blockIdx.y +
    threadIdx.y);
```

```
    int j0 = blockDim.x * blockIdx.x + threadIdx.x;
    BASE_TYPE sum = 0;

    for (int k = 0; k < Acols; k++)
      sum += A[i0 + k] * B[k * Bcols + j0];

    int ind = Bcols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

    C[ind] = sum;
}
```

Листинг кода:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

#define BLOCK_SIZE 16
// тип, который будут иметь элементы матриц
#define BASE_TYPE double
// функция перемножения матриц
__global__ void matrixMult(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int Acols, int Bcols)
{
    int i0 = Acols * (blockDim.y * blockIdx.y +
    threadIdx.y);
    int j0 = blockDim.x * blockIdx.x + threadIdx.x;
    BASE_TYPE sum = 0;

    for (int k = 0; k < Acols; k++)
      sum += A[i0 + k] * B[k * Bcols + j0];

    int ind = Bcols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

    C[ind] = sum;
}
```

```
int toMultiple(int a, int b) {

  int mod = a % b;

  if (mod != 0) {
    mod = b - mod;
    return a + mod;
  }
  return a;
}

int main()
{
  //start, stop - for Kernel time
    cudaEvent_t start, stop;
  cudaEventCreate(&start);
  cudaEventCreate(&stop);

    // количество строк и столбцов матрицы
    int Arows = 100;
    int Acols = 200;
    int Brows = Acols;
    int Bcols = 150;

    Arows = toMultiple(Arows, BLOCK_SIZE);
    printf("Arows = %d\n", Arows);

    Acols = toMultiple(Acols, BLOCK_SIZE);
    printf("Acols = %d\n", Acols);

    Brows = toMultiple(Brows, BLOCK_SIZE);
    printf("Brows = %d\n", Brows);

    Bcols = toMultiple(Bcols, BLOCK_SIZE);
    printf("Bcols = %d\n", Bcols);

    size_t Asize = Arows * Acols * sizeof(BASE_TYPE);
    size_t Bsize = Brows * Bcols * sizeof(BASE_TYPE);
    size_t Csize = Arows * Bcols * sizeof(BASE_TYPE);
```

```
  BASE_TYPE *h_A = (BASE_TYPE *)malloc(Asize);
  BASE_TYPE *h_B = (BASE_TYPE *)malloc(Bsize);
  BASE_TYPE *h_C = (BASE_TYPE *)malloc(Csize);

  for (int i = 0; i < Arows * Acols; ++i) {
      h_A[i] = rand()/(BASE_TYPE)RAND_MAX;
  }
  for (int i = 0; i < Brows * Bcols; ++i) {
      h_B[i] = rand()/(BASE_TYPE)RAND_MAX;
  }

  BASE_TYPE *d_A = NULL;
  cudaMalloc((void **)&d_A, Asize);

  BASE_TYPE *d_B = NULL;
  cudaMalloc((void **)&d_B, Bsize);

  BASE_TYPE * d_C = NULL;
  cudaMalloc((void **)&d_C, Csize);
  cudaMemcpy(d_A, h_A, Asize, cudaMemcpyHostToDevice);

  cudaMemcpy(d_B, h_B, Bsize, cudaMemcpyHostToDevice);

  dim3 threadsPerBlock = dim3(BLOCK_SIZE, BLOCK_SIZE);
  dim3 blocksPerGrid = dim3(Bcols / BLOCK_SIZE, Arows /
  BLOCK_SIZE);

  cudaEventRecord( start, 0);

  matrixMult<<<blocksPerGrid, threadsPerBlock>>>(d_A,
  d_B, d_C, Acols, Bcols);

  cudaEventRecord( stop, 0);
cudaEventSynchronize( stop );
float KernelTime;
cudaEventElapsedTime( &KernelTime, start, stop);
  printf("KernelTime: %.2f milliseconds\n", Ker-
  nelTime);
```

```
    cudaMemcpy(h_C, d_C, Csize, cudaMemcpyDeviceToHost);

    printf("Test STARTED\n");
    for (int i = 0; i < Arows; i++) {
    for (int j = 0; j < Bcols; j++) {
        BASE_TYPE sum = 0;
        for (int k = 0; k < Acols; k++)
                sum += h_A[i * Acols + k] * h_B[k *
                Bcols + j];

            if (fabs(h_C[i * Bcols + j] - sum) > 1e-3)
            {
                fprintf(stderr, "Result verification
                failed at element [%d, %d]!\n", i, j);
                printf("sum = %f, h_C[i * Bcols + j] =
                %f\n", sum, h_C[i * Bcols + j]);
                 exit(EXIT_FAILURE);
            }
          }
    }
    printf("Test PASSED\n");

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C);
    cudaEventDestroy( start );
  cudaEventDestroy( stop );

  return 0;
}
```

Контрольные вопросы

1. Чем отличается создание матрицы на GPU от создания матрицы на CPU и последующего копирования ее элементов на GPU?

2. В каких участках кода при операциях над матрицами возникают потери в производительности?

Лабораторная работа

1. Проверьте, является ли данная квадратная матрица ортогональной, путем умножения на транспонированную (не выделяя под нее дополнительной памяти) и сравнения результата с единичной матрицей.

Домашняя работа

1. Проверьте, являются ли две заданные квадратные матрицы A и B коммутирующими (т.е. их произведение не зависит от порядка умножения: AB = BA).

2. Напишите функцию для сложения двух матриц M×N, заданных как вектор векторов (float a[M][N], b[M][N]). Матрицы задаются на хосте, после копируются на девайс, и там производится сложение.

Глава 7. Типы памяти. Разделяемая память

В CUDA предусмотрено несколько типов памяти, каждый из которых предназначен под определенные нужды. Ниже приведена таблица типов памяти GPU.

Таблица 3

Типы памяти GPU

Тип памяти	**Доступ**	**Выделяется**	**Скорость работы**
Регистры	чтение/запись	на поток	Высокая
Локальная	чтение/запись	на поток	Низкая
Разделяемая	чтение/запись	на блок	Высокая
Глобальная	чтение/запись	на все решетки	Низкая
Константная	чтение	на все решетки	Низкая
Текстурная	чтение	на все решетки	Низкая

Регистры используется для хранения локальных переменных. Скорость доступа для данного типа памяти самая быстрая, и выделение происходит на каждую нить. **Локальная память** используется для хранения локальных переменных, когда регистров не хватает, скорость доступа низкая. Память выделяется так же, как и для регистров. **Разделяемая память** используется для хранения массивов данных, используемых совместно всеми нитями в блоке. Эта память имеет чуть меньшую скорость доступа, чем регистры, и выделяется на блок. В **глобальной памяти** хранятся исходно все данные, это общая память для всех блоков. **Константная память** используется для передачи параметров в ядро, превышающих допустимые размеры для параметров ядра, выделение целиком на решетку. **Текстурная память** используется для хранения больших массивов данных, выделяется целиком на решетку.

Обращение к памяти в CUDA осуществляется одновременно из половины количества нитей в блоке. Все типы памяти, кроме регистровой и локальной, можно использовать как эффективно, так и неэффективно. Поэтому необходимо четко понимать их особенности и области применения.

Рассмотрим более подробно разделяемую память. Основной особенностью разделяемой памяти, является то, что она оптимизирована под обращение к ней одновременно половины количества нитей в блоке (16 нитей). Вся разделяемая память разбита на 32 банка, состоящих из 32 битовых слов, причем, последовательно идущие слова попадают в последовательно идущие блоки. Обращение к каждому банку происходит независимо. Второй важной особенностью разделяемой памяти является то, что она выделяется на блок. Для задания разделяемой памяти используется спецификатор ***__shared__***. Часто использование разделяемой памяти требует явной синхронизации внутри блока.

При работе с разделяемой памятью необходимо обеспечить синхронизацию потоков, что в CUDA довольно просто – она предоставляет примитивный барьер, функцию __syncthreads(). При ее вызове потоки приостанавливаются до того момента, пока все потоки не достигнут этой точки. Важно, чтобы вызов __syncthreads() у всех потоков находился в одном и том же месте (в частности, был безусловным, т. е. не находился внутри оператора if и т.п.), в противном случае результаты могут быть непредсказуемыми, в частности, может возникнуть «мертвая блокировка» (deadlock).

Приведем фрагмент кода создания массива данных в разделяемой памяти.

```
#define BS 256 //Размер блока

__global__ void Kernel (float* data)
{
// Создание массива в разделяемой памяти
  __shared__ float a[BS];
  int i = blockIdx.x * BS + threadIDx.x;
// Копирование из глобальной памяти в разделяемую
  a[i] = data[i];
```

```
// Синхронизируем нити.
// Перед использованием нужно быть уверенным, что все
данные скопированились
  __syncthreads();
// Использонание данных разделяемой памяти
}
```

Данный пример демонстрирует только работу с разделяемой памятью. Для того, чтобы понять, как же разделяемая память помогает в ускорении работы, напишем программу для вычисления скалярного произведения векторов. Попробуем написать эту программу с использованием глобальной памяти. Используя механизм событий, замерим время работы ядра.

Рассмотрим, как изменится ядро, если вместо глобальной памяти использовать разделяемую память.

```
__global__ void scalMult(const BASE_TYPE *A, const
BASE_TYPE *B, BASE_TYPE *C, int numElem)
{
// Переменная для хранения суммы элементов
    BASE_TYPE sum = 0.0;
// Создание массивов в разделяемой памяти
  __shared__ BASE_TYPE ash[BLOCK_SIZE];
  __shared__ BASE_TYPE bsh[BLOCK_SIZE];
// Копирование из глобальной памяти
    ash[threadIdx.x] = A[blockIdx.x * blockDim.x +
    threadIdx.x];
    bsh[threadIdx.x] = B[blockIdx.x * blockDim.x +
    threadIdx.x];
// Синхронизация нитей
  __syncthreads();
// Вычисление скалярного произведения
  if (threadIdx.x == 0)
  {
    sum = 0.0;
    for (int j = 0; j < blockDim.x; j++)
      sum += ash[j] * bsh[j];
```

```
    C[blockIdx.x] = sum;
  }
}
```

Заменим ядро программы и оценим время работы ядра, использующего разделяемую память. Так как каждая нить теперь работает с памятью, чья скорость работы более высокая, большую часть времени работы программы занимают вычисления, а не доступ к памяти. Более наглядно это можно наблюдать в случае перемножения матриц.

Банки данных

Доступ к памяти осуществляется через 32 банка данных. Причем номер банка можно посчитать по формуле

Номер банка = (Адрес в байтах / 4) % 32.

Каждый банк в определенный момент времени обрабатывает одну транзакцию (чтение или запись). Оптимальный доступ к данным обеспечивается, когда каждая нить последовательно соотносится со своим банком (см. рис. 6а).

Также конфликта между банками нет, если каждой нити в любом порядке соответствует свой банк. Пример такого доступа изображен на рис. 6б. Конфликт имеет место, когда несколько нитей пытаются обратиться к одному банку данных, как изображено на рис. 6в. Однако в случае, когда все нити записывают или считывают с одного банка, конфликта не происходит.

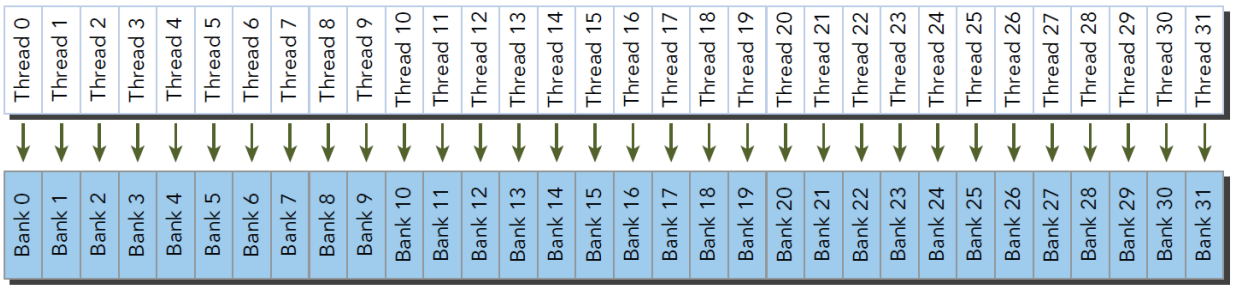


Рис. 6а. Оптимальный доступ к банкам данных

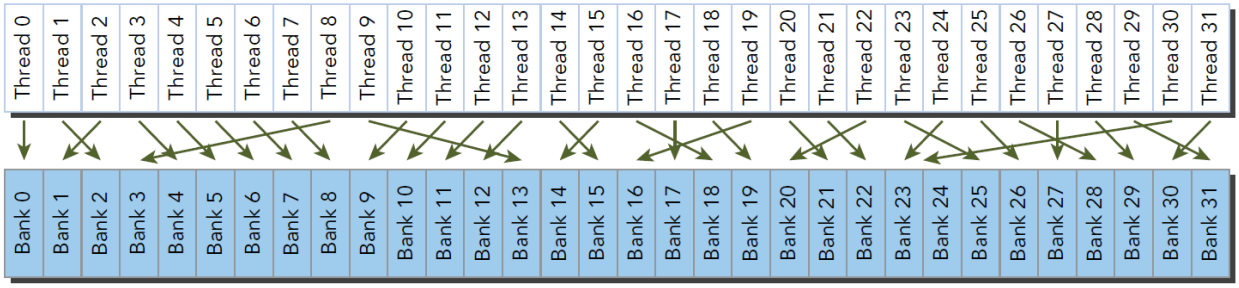


Рис. 6б. Бесконфликтный доступ к банкам данных

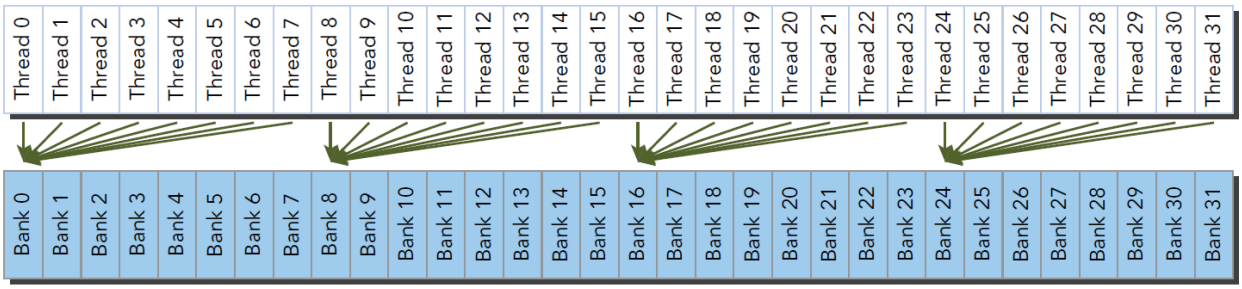


Рис. 6в. Конфликтный доступ к банкам данных

Рассмотрим наиболее распространенные случаи, когда могут иметь место конфликты банков данных. В следующем примере

```
__shared__ char s[N];
char data = s[threadIdx.x];
```

имеем однобайтовый массив, и, так как в одном банке содержится четыре байта, одновременно в один банк будут обращаться четыре нити. Это приведет к конфликту четвертого порядка (4 нити на 1 банк). Отметим, что на рис. 6в приведен пример конфликтов шестого порядка.

Один из способов решения такого конфликта состоит в добавлении избыточных данных:

```
__shared__ char s[4 * N];
char data = s[threadIdx.x * 4];
```

В следующем примере будет конфликт второго порядка (`short` – двухбайтовый тип)

```
__shared__ short s[N];
short data = s[threadIdx.x];
```

Чаще возникают конфликты, когда используются четырехбайтовые числа и номер в массиве вычисляется по формуле. Например, в следующем коде:

```
__shared__ float s[N];
float data = s[threadIdx.x * L];
```

при четных `L` возникают конфликты. Действительно, при `L` = 2 нулевая нить и 16-натая нить обращаются в нулевой банк, а при `L` = 4 уже нити с номерами 0, 8, 16 и 24 обращаются в нулевой банк, создавая конфликт четвертого порядка.

Пример. Оптимизация программы по перемножению двух матриц

Пусть даны две матрицы размером $N * N$ (для удобства будем считать, что N кратно 16). Если решать данную задачу, получая данные постоянно из глобальной памяти, то понадобится $2N$ чтений из глобальной памяти. Становится ясно, что основным тормозящим программу фактором является не что иное, как чтение из глобальной памяти, которая имеет низкую скорость передачи.

Заметно ускорить программу поможет правильная организация вычислений и разделяемая память. Обратим внимание на то, что для вычисления элемента новой матрицы необходимы лишь небольшие части исходных матриц. Для повышения быстродействия программы разобьем новую матрицу на подматрицы таким образом, чтобы обработкой данной подматрицы занимался только один блок. В данной задаче разобьем на подматрицы (C') размером 16*16 элементов (см. рис. 7).

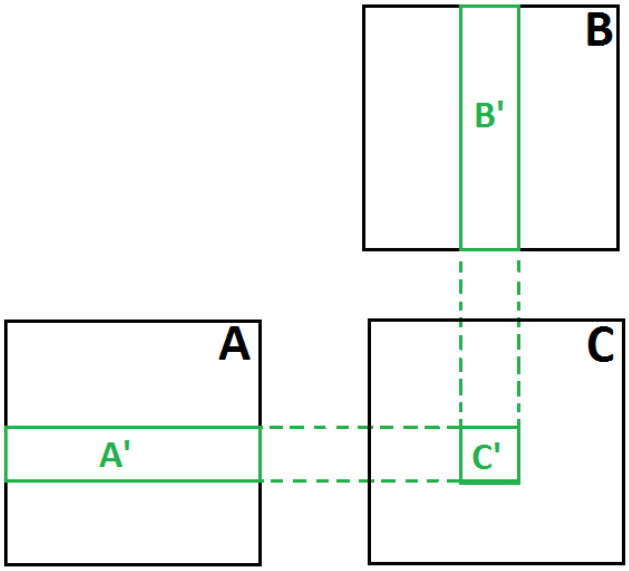


Рис. 7. Вычисление подматрицы новой матрицы

Однако, из-за малого объема разделяемой памяти, подматрицы A' и B' не могут быть целиком скопированы. Поступим с A' и B' так же, как поступили с подматрицей C', разобьем каждую матрицу на подматрицы размером 16*16. В итоге вычисление C' будем проводить в $N/16$ шагов (см. рис. 8).

Вычисление подматрицы C' можно записать следующим образом:

$$C' = A'_1 * B'_1 + A'_2 * B'_2 + \cdots + A'_{\frac{N}{16}} * B'_{\frac{N}{16}}.$$

Теперь на каждом шаге будем подгружать в разделяемую память по одной подматрице A'_i и B'_i, вычислять соответствующую им сумму для элементов произведения и по окончании всех вычислений записывать в результирующую матрицу.

Укажем в начале программы размер блока и тип элементов массива:

```
// размер блока или размер подматрицы
#define BLOCK_SIZE 16
//тип, который будут иметь элементы матриц
#define BASE_TYPE double
```

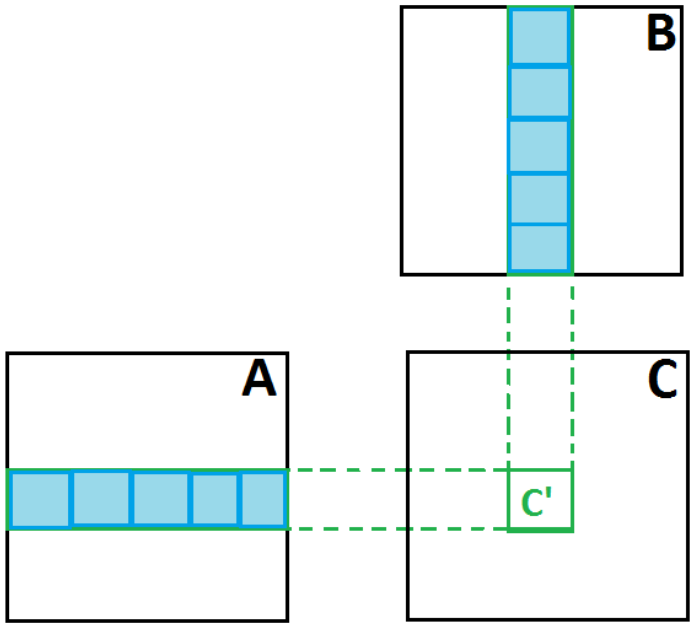


Рис. 8. Разбиение исходных матриц

С учетом всего вышесказанного об организации вычислений, ядро будет выглядеть следующим образом:

```
__global__ void matrixMult(const BASE_TYPE *A, const
 BASE_TYPE *B, BASE_TYPE *C, int Acols, int Bcols)
{
// индекс начала первой подматрицы A, которую
// обрабатывает блок
     int aBegin = Acols * blockDim.y * blockIdx.y;
 // индекс конца подматрицы A, которую обрабатывает блок
     int aEnd = aBegin + Acols - 1;
// шаг для перебора подматриц A
     int aStep = blockDim.x;
// индекс начала первой подматрицы B, которую
// обрабатывает блок
   int bBegin = blockDim.x * blockIdx.x;
// шаг для перебора подматриц B
     int bStep = blockDim.y * Bcols;

 // Выделение разделяемой памяти для подматриц
     __shared__ BASE_TYPE as [BLOCK_SIZE][BLOCK_SIZE];
     __shared__ BASE_TYPE bs [BLOCK_SIZE][BLOCK_SIZE];
```

```
// переменная для вычисления элемента подматрицы
    BASE_TYPE sum = 0.0;

    for (int ia = aBegin, ib = bBegin; ia < aEnd; ia +=
    aStep, ib += bStep)
    {
// загрузка подматриц A и B из глобальной памяти в
// разделяемую
        as[threadIdx.y][threadIdx.x] = A[ia + Acols *
        threadIdx.y + threadIdx.x];
        bs[threadIdx.y][threadIdx.x] = B[ib + Bcols *
        threadIdx.y + threadIdx.x];

// синхронизация нитей
      __syncthreads();

// перемножение двух матриц
      for (int k = 0; k < blockDim.x; k++)
            sum += as[threadIdx.y][k] *
            bs[k][threadIdx.x];

// синхронизация нитей
      __syncthreads();
    }

// индекс результирующего элемента в глобальной памяти
    int ind = Bcols * (blockDim.y * blockIdx.y +
    threadIdx.y) + blockDim.x * blockIdx.x + threadIdx.x;

// запись элемента в глобальную память
    C[ind] = sum;
}
```

После любой работы с разделяемой памятью необходимо синхронизировать нити. В данном примере важно произвести синхронизацию до момента начала расчетов, чтобы убедиться в том, что все данные загружены, а также после расчетов, чтобы убедиться в том, что текущие подматрицы не нужны и можно подгружать новые подматрицы.

Вызов ядра из главной функции:

```
// параметры запуска ядра
dim3 threadsPerBlock = dim3(BLOCK_SIZE, BLOCK_SIZE);
dim3 blocksPerGrid = dim3(Bcols / BLOCK_SIZE, Arows /
BLOCK_SIZE);

// старт времени работы ядра
cudaEventRecord( start, 0);

// вызов ядра
matrixMult<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B,
d_C, Acols, Bcols);

// стоп времени работы ядра
cudaEventRecord( stop, 0);
cudaEventSynchronize( stop );

// вычисление времени работы
float KernelTime;
cudaEventElapsedTime( &KernelTime, start, stop);
```

Использование такого алгоритма для перемножения двух матриц требует 2 * N / 16 чтений из глобальной памяти. Понятно, что правильное использование разделяемой памяти может поднять быстродействие программы даже более чем на порядок. Также метод вычисления суммы произведений элементов по блокам, который был применен в ядре для вычисления произведения двух матриц, ускоряет вычисления и часто применяется в подобных задачах.

Если размер разделяемой памяти не задан изначально и определяется в процессе работы программы, то необходимо его указывать при вызове ядра

```
myKernelFunc<<<gridSize, blockSize,
sharedMemSize>>>(float *param1, int *param2)
```

где `sharedMemSize` – размер дополнительной памяти, выделяемой при запуске ядра,

Контрольные вопросы

1. Какие типы памяти выделяют в технологии CUDA?

2. Какой спецификатор применяется для задания разделяемой памяти? Перечислите основные особенности разделяемой памяти.

Лабораторная работа

1. Напишите программу для скалярного умножения двух векторов, используя разделяемую память. Разделяемая память выделяется с учетом ограничения ее размеров на блок. Количество нитей при вызове ядра равно числу элементов массива с разделяемой памятью. Замерьте время работы ядра программы.

2. Напишите программу вычисления определенного интеграла функции одной переменной по квадратурной формуле центральных прямоугольников.

Домашняя работа

1. Доработайте программу вычисления определенного интеграла, используя другие квадратурные формулы: формулы трапеций и Симпсона. Сравните точность результатов.

2. Напишите программу вычисления евклидовой нормы вектора (путем скалярного умножения его на себя и вычисления корня из произведения).

Глава 8. Константая и текстурная память

Помимо использования разделяемой памяти существует еще два вида памяти, которые можно использовать в CUDA – это константная и текстурная память. Эти виды памяти являются медленнее разделяемой, но за счет кэширования они превосходят по скорости глобальную память.

Константная память

Константная память используется тогда, когда в ядро необходимо передать много различных данных, которые будут одинаково использоваться всеми нитями ядра. Эта память выделяется в глобальной памяти в пределах 64 кБ. При этом кэширование происходит отдельного для каждого мультипроцессора, размер кэша равен 8 кБ для всех версий производительности кроме 6.0, для 6.0 объем кэша составляет 4 кБ.

Объявление переменной делается глобально, используя слово `__constant__`. Пример использования константного массива:

```
__device__ __constant__ float contsData[4];
float hostData[4];
……
// присваивание значений в hostData
// копирование данных с центрального процессора в
// константную память
cudaMemcpyToSymbol (constData, hostData, 4*sizeof(float),
0, cudaMemcpyHostToDevice);
```

Слово `__device__` перед `__constant__` можно опускать, так как по умолчанию константная память выделяется на графическом устройстве. Использование внутри ядра константной памяти ничем не отличается от использования любой глобальной переменной. Рассмотрим следующий пример использования константы:

```
__constant__ int add;
```

```
// функция сложения двух векторов с прибавлением
// постоянной add
__global__ void addKernel(int *c, const int *a, const int
*b)
{
    int i = threadIdx.x;
  c[i] = a[i] + b[i] + add;
}
……
// инициализация и вызов функции сложения в основной
// программе, size – размер массивов
int r = 2;
cudaMemcpyToSymbol(add, &r, sizeof(int), 0,
cudaMemcpyHostToDevice);

addKernel<<<1, size>>>(dev_c, dev_a, dev_b);
```

Текстурная память

При рассмотрении разделяемой памяти мы предполагали, что использование памяти внутри блока сильно локализовано, то есть блоку для его работы требуется лишь какой-то определенный участок массива; если же локализовать обращение не удается, а скорость обращения к памяти является узким местом программы, можно попробовать использовать текстурную память.

Основная особенность текстурной памяти – это использования кэша. Однако существуют дополнительные стадии конвейера (преобразование адресов, фильтрация, преобразование данных), которые снижают скорость первого обращения. Поэтому текстурную память разумно использовать с следующих случаях:

- объем данных не влезает в shared память;
- паттерн доступа хаотичный;
- данные переиспользуются разными потоками.

Для использования текстурной памяти необходимо задать объявление текстуры как глобальную переменную:

```
texture < type, dim, tex_type> g_TexRef ;
```

Type – тип хранимых переменных, dim – размерность текстуры (1, 2, 3) Tex_type – тип возвращаемых значений

- cudaReadModeNormalizedFloat,
- cudaReadModeElementType.

Кроме того, для более полного использования возможностей текстурной памяти можно задать описание канала:

```
struct cudaChannelFormatDesc
{
int x , y , z , w;
enum cudaChannelFormatKind f ;
} ;
```

Задает формат возвращаемого значения.

int x, y, z, w – числа в интервале [0, 32], проекция исходного значения по битам

cudaChannelFormatKind – тип возвращаемого значения

- cudaChannelFormatKindSigned – знаковые int
- cudaChannelFormatKindUnsigned – беззнаковые int
- cudaChannelFormatKindFloat – float

Текстурная память, как и константная, кэшируется. Причем, размер кэша зависит от производительности. До производительности 6.0 размер кэша изменяется от 12 до 48 КБ, для 6.x – от 24 до 48 КБ, для 7.0 – от 32 до 128 КБ.

Примеры использования одномерной текстуры

Приведем пример программы для работы с одномерной текстурой, привязанной к CUDA массиву.

```
#include "cuda_runtime.h"
```

```
#include "device_launch_parameters.h"
#include <stdio.h>

// объявляем одномерный текстурный объект
texture<float, 1, cudaReadModeElementType> texRef;

__global__ void readTexels(int n, float *d_out)
{
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < n)
    {
    float x = tex1D(texRef, float(idx));
    d_out[idx] = x;
  }
}

#define NUM_THREADS 256

int main()
{
  int N = 10;
    int nBlocks = N / NUM_THREADS + ((N % NUM_THREADS) ?
    1 : 0);
  float *d_out;

  // выделяем память на GPU
  cudaMalloc((void**)&d_out, sizeof(float) * N);

  // выделяем память на CPU
  float *h_out = (float*)malloc(sizeof(float)*N);

  // заполняем массив data
    float *data = (float*)malloc(N * sizeof(float));

    for (int i = 0; i < N; i++) data[i] = float(2 * i);

    // создаем CUDA массив на device
    cudaArray* cuArray;
    cudaMallocArray(&cuArray, &texRef.channelDesc, N, 1);
```

```
    cudaMemcpyToArray(cuArray, 0, 0, data,
    sizeof(float)*N, cudaMemcpyHostToDevice);

    // привязываем текстуру к CUDA массиву
    cudaBindTextureToArray(texRef, cuArray);

    // устанавливаем атрибуты текстуры
    texRef.normalized = false;
    texRef.filterMode = cudaFilterModePoint;

    readTexels << <nBlocks, NUM_THREADS >> >(N, d_out);

    // копируем с девайса на хост
    cudaMemcpy(h_out, d_out, sizeof(float)*N, cudaMem-
    cpyDeviceToHost);

    for (int i = 0; i < N; i++)
    printf("%f\n", h_out[i]);


    free(h_out);
    cudaFree(d_out);
    cudaFreeArray(cuArray);
    cudaUnbindTexture(texRef);
    getchar();
}
```

Программа выделяет память на GPU, привязывает к ней текстуру. Затем копирует из текстуры в массив. Ответ должен быть следующим:

```
0.000000
2.000000
4.000000
6.000000
8.000000
10.000000
12.000000
14.000000
16.000000
18.000000
```

Следующий пример кода связывает текстуру с линейной памятью. Эта программа заполняет на девайсе обыкновенный массив из текстуры, после этого копирует его на хост и сравнивает с исходным массивом. Программа должна выдать сообщение «Массивы совпали».

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <assert.h>

void checkCUDAError(const char *msg)
{
  cudaError_t err = cudaGetLastError();
  if (cudaSuccess != err)
    {
    fprintf(stderr, "Cuda error: %s: %s.\n", msg,
cudaGetErrorString(err));
    exit(EXIT_FAILURE);
  }
}

texture<float, 1, cudaReadModeElementType> texRef;

__global__ void kernel(int n, float *d_out)
{
  int idx = blockIdx.x*blockDim.x + threadIdx.x;
  if (idx < n)
        d_out[idx] = tex1Dfetch(texRef, idx);
}

#define NUM_THREADS 256
int main()
{
  int N = 5000;
    int nBlocks = N / NUM_THREADS + ((N % NUM_THREADS) ?
    1 : 0);
  int memSize = N * sizeof(float);
```

```
  // data fill array with increasing values
  float *data;
  data = (float*)malloc(memSize);
  for (int i = 0; i < N; i++) data[i] = float(i);

  float *d_a;
  cudaMalloc((void **)&d_a, memSize);
    cudaMemcpy(d_a, data, memSize, cudaMemcpyHostToDe-
    vice);

  cudaBindTexture(0, texRef, d_a, memSize);
  checkCUDAError("bind");

  kernel << <nBlocks, NUM_THREADS >> >(N, d_a);

  float *h_out = (float*)malloc(memSize);
    cudaMemcpy(h_out, d_a, memSize, cudaMemcpyDeviceTo-
    Host);
  checkCUDAError("cudaMemcpy");

  for (int i = 0; i << N; i++)
    assert(data[i] == h_out[i]);

  printf("Массивы совпали \n");

  cudaUnbindTexture(texRef);
  checkCUDAError("cudaUnbindTexture");
  free(h_out);
  free(data);
  getchar();
}
```

Контрольные вопросы

1. Где выделяется константная и текстурная память?

2. Где расположен кэш для константной и текстурной памяти? Каковы размеры кэша?

Лабораторная работа

1. Напишите программу для скалярного умножения двух векторов, используя константную память. Замерьте время работы ядра программы.

2. Напишите программу вычисления определенного интеграла функции одной переменной по квадратурной формуле центральных прямоугольников. Используйте тектстурную память, привязанную к линейной памяти.

Домашняя работа

1. Напишите программу для скалярного умножения двух векторов, расположенных в текстурной памяти. Испольуйте три алгоритма: 1) в обоих случаях привязка к CUDA массиву; 2) в одном – к CUDA массиву, в другом – к линейной памяти; 3) в обоих случаях привязаны к линейной памяти. Сравните время работы для различных размеров массивов.

Литература

1. *Боресков А.В.* Основы работы с технологией CUDA / А.В. Боресков, А.А. Харламов. – М.: ДМК Пресс, 2010. – 232 с.

2. *Сандерс Дж.* Технология CUDA в примерах: Введение в программирование графических процессоров / Дж. Сандерс, Э. Кэндрот. – М.: ДМК Пресс, 2013. – 232 с.

3. *Cheng J.* Professional CUDA C Programming / J. Cheng, M. Grossman, T. McKercher. – Indianapolis: John Wiley & Sons, Inc., 2014. – 528 p.

Электронное учебное издание
сетевого распространения

Тумаков Д.Н.

Гиниятова Д.Х.

Гуськов В.С.

ПРОГРАММИРОВАНИЕ ГРАФИЧЕСКИХ УСТРОЙСТВ. ТЕХНОЛОГИЯ CUDA

Учебное пособие

Подписано к использованию 21.11.2022.
Формат 60×84 1/16. Гарнитура «Times New Roman».
Усл. печ. л. 5,0. Заказ 118/11

Издательство Казанского университета
420008, г. Казань, ул. Профессора Нужина, 1/37
тел. (843) 206-52-14 (доп. 1705), 206-52-14 (доп. 1704)